Projet de Data Science

Prédiction des prix de logement

Étude réalisée à partir du jeu de données Ames Housing



Début du projet : Mai 2026

Fin du projet : Juillet 2026

Nom et prénom :	Ngomo Luigy
Niveau d'études :	Master 2
Diplôme :	Mathématiques appliquées, statistiques et Data Science
Dernière expérience professionnelle :	Enseignant de Mathématiques et d'Algorithmique Appliquée
Profil LinkedIn :	www.linkedin.com/in/luigy-ngomo-7129312a6
Adresse électronique :	ngomoluigy@gmail.com

Résumé

Ce projet porte sur la prédiction du prix de vente des maisons à partir du jeu de données *Ames Housing*. Dans cette étude, nous construisons, comparons et interprétons les résultats de plusieurs modèles prédictifs capables d'estimer le prix de vente d'une maison en fonction de ses caractéristiques physiques, géographiques et qualitatives.

Le travail comprend notamment le traitement des valeurs manquantes, le traitement des valeurs aberrantes, l'analyse des corrélations, l'encodage des variables catégorielles, la création de nouvelles variables, la correction de l'asymétrie des variables, l'entraînement de plusieurs modèles de machine learning et la comparaison de leur performance sur un ensemble de validation indépendant, etc.

Les modèles étudiés sont notamment : les arbres de décisions de régression, les forets aléatoires, les arbres extrêmement aléatoires, le gradient boosting, l'Extrême Gradient Boosting (XGBoost) et la régression Ridge.

Mots et expressions importantes : Variable cible, Ames Housing, régression, machine learning, Random Forest, variables explicatives, Decision Tree Regressor (CART), ensemble de données d'entrainement, Extreme Gradient Boosting (XGBoost), validation croisée K-fold, stacking, ensemble de données de validation, Régréssion Ridge, tuning d'hyperparamètres, Extra Trees, Feature Engineering, Ordinal Encoding, Gradient Boosting, skewness, One-Hot Encoding, RMSE (Root Mean Square Error), RMSLE (Root Mean Square Logarithmic Error), transformée de Box-Cox, optuna, méta-modèle, prédictions out-of-fold, MAE (Mean Absolute Error), R^2 (coefficient de détermination), bootstrap, etc.

Table des matières

Résumé **3**

I Introduction et objectifs **10**

II Chargement des données **10**

III Familiarisation avec les données **11**

IV Analyse exploratoire et prétraitement des données **13**

IV.1 Distribution de la variable cible SalePrice 13

IV.1.1 Coefficient d'asymétrie 13

IV.1.2 Transformation logarithmique 15

IV.2 Gestions des valeurs manquantes 18

IV.2.1 PoolQC 19

IV.2.2 MiscFeature 20

IV.2.3 MasVnrType et MasVnrArea 21

IV.2.4 FireplaceQu 23

IV.2.5 LotFrontage 23

IV.2.6 Alley, Fence 24

IV.2.7 GarageType & GarageYrBlt & GarageFinish & GarageQual & GarageCond 25

IV.2.8 BsmtExposure & BsmtFinType2 & BsmtQual & BsmtCond & BsmtFinType1 26

IV.2.9 Electrical 28

IV.3 Corrélations 28

IV.4 Gestion des valeurs aberrantes 30

IV.4.1 Valeurs aberrantes des variables numériques quantitatives fortement corrélées avec SalePrice 30

IV.4.2 Valeurs aberrantes des autres variables quantitatives 32

IV.5 Gestion des variables catégorielles (Encodage) 38

IV.5.1 Ordinal Encoding 38

IV.5.2 One-Hot Encoding 39

IV.5.3 Mise en oeuvre 40

V Entrainement des modèles (Random Forest Regressor, Extra Trees Regressor, Decision Tree Regressor (CART), Gradient Boosting Regressor, Extreme Gradient Boosting (XGBoost), Ridge Regression) et évaluation sans creation de nouvelles variables et correction d'asymétrie. 41

V.1 Arbres de décisions (decision trees) . . . 42

V.1.1 Définition . . . 42

V.1.2 Les arbres de classification . . . 42

V.1.3 Les arbres de régression . . . 43

V.1.4 Principe de fonctionnement d'un arbre de régression . . . 43

V.2 Foret Aléatoire (Random Forest) . . . 50

V.2.1 Définition . . . 50

V.2.2 Structure . . . 50

V.2.3 Problème que peut résoudre une foret aléatoire . . . 51

V.2.4 Principe de construction des arbres d'une foret aléatoire . . . 51

V.2.5 La prédiction finale . . . 52

V.3 Entrainement des modèles . . . 55

V.3.1 Validation croisée K-Fold . . . 55

V.3.2 Entrainement des modèles de bases (avec hyperparamètres choisis manuellement) . . . 57

VI Création de nouvelles variables (Feature Creation) 60

VI.1 Feature Creation . . . 60

VI.2 Analyse des corrélations . . . 62

VII Correction de l'asymétrie des variables numériques quantitatives continues 64

VII.1Variables asymétriques . . . 64

VII.2 Correction d'asymétrie : transformation de Box-Cox . . . 66

VIII Entrainement des modèles (Random Forest Regressor, Extra Trees Regressor, Decision Tree Regressor (CART), Gradient Boosting Regressor, Extreme Gradient Boosting (XGBoost), Ridge Regression) et évaluation après creation de nouvelles variables et correction d'asymétrie. 70

VIII.1 Résultats des modèles entrainés et évalués (par cv) après correction d'asymétrie et création de nouvelles variables . . . 70

VIII.2 Comparaison avec les modèles entainées et évalués (par cv) avant correction d'asymétrie et création de nouvelles variables . . . 71

IX Optimisation des hyperparamètres **72**

IX.1 Principe de fonctionnement d'Optuna 72

IX.1.1 La fonction objectif 73

IX.1.2 Optuna 73

X Stacking et évaluation finale des modèles **79**

X.1 Choix des modèles de base 79

X.2 Obtention des valeurs prédites par validation croisée 79

X.3 Construction du nouveau dataset 79

X.4 Entraînement du méta-modèle 80

X.5 Réentraînement des modèles de base et prédiction d'une nouvelle maison 81

XI Évaluation finale des modèles sur l'ensemble de validation **82**

XI.1 Classement final sur l'ensemble de validation 82

XI.2 Histogramme des prix réels et des prix prédits sur l'ensemble de validation 83

XI.3 Nuage de points des prix réels et des prix prédits sur l'ensemble de validation . . 84

XII Importance des variables **85**

Références **87**

Table des figures

1 Extrait des dix premières observations du jeu de données Ames Housing. 10

2 Histogramme de la distribution des prix des maisons et QQ-plot associé. 14

3 Fonction logarithme népérien 15

4 Histogramme de la distribution des prix des maisons transformées et QQ-plot associé. 16

5 Résumer sur la distribution de la variable cible 17

6 Valeurs prises par la variable PoolArea 19

7 Les deux maisons avec MiscVal nulle et MiscFeature renseigné 21

8 Les 8 valeurs manquantes de la variable MasVnrArea 21

9 Les deux maisons avec MasVnrArea nulle et MasVnrType renseigné 22

10 Maisons avec TotalBsmtS > 0 et BsmtExposure ou BsmtFinType2 manquantes 27

11 Top 15 des variables les plus corrélées avec SalePrice 29

12 Distributions des variables les plus fortement corrélées (>0.5) avec SalePrice . . 30

13 Nuages de points de GarageArea TotalBsmtSF, 1stFlrSF et GrLivArea avec SalePrice 31

14 Distribution des variables quantitatives 1 33

15 Distribution des variables quantitatives 2 34

16 Distribution des variables quantitatives 3 35

17 Nuages de points de LotFrontage, LotArea, BsmtFinSF1, EnclosedPorch, MiscVal et GarageYrBlt avec SalePrice 36

18 Autres caractéristiques des maisons avec des potentielles valeurs aberrantes . . . 37

19 Léger aperçu du One-hot-Encoding sur la variable Neighborhood 41

20 Exemple d'arbre de décision de classification simple 42

21 Principe général d'une forêt aléatoire. 50

22 Schéma résumant les étapes principales d'un random forest 52

23 Classement des modèles 59

24 Aperçu des nouvelles variables crées 62

25 Top 20 des variables les plus corrélées avec SalePrice 63

26 Asymétrie des variables numériques quantitatives continues 65

27 Distributions avant et après correction (premier groupe de variable) 68

28 Distributions avant et après correction (second groupe de variable) 69

29 Classement des modèles après correction de skewness et feature creation 70

30 Tableau de comparaison des performanes des modèles avant et après correction d'asymétrie et création de nouvelles variables 71

31 Comparaison des performanes des modèles avant et après correction d'asymétrie et création de nouvelles variables 71

32 Temps d'exécution du tuning d'hyperparamètres pour chaque modèle 75

33 Tableau des erreurs des modèles (rmsle) avant et après tuning 77

34 Comparaison des erreurs des modèles (rmsle) avant et après tuning 78

35 Aperçu de quelques prédictions (logarithmiques) des modèles de base 80

36 Recherche du meilleur paramètre alpha pour le méta-modèle 81

37 Valeurs réelles des coefficients appris par le méta-modèle lors de l'entraînement . 81

38 Classement final des modèles sur l'ensemble de validation 82

39 Distribution des prix réels et des prix prédits pour les différents modèles 83

40 Nuages de points des valeurs réelles et des valeurs prédites pour les différents modèles 84

41 Importance des variables 85

Liste des tableaux

1 Interprétation classique du coefficient d'asymétrie (skewness). 14
2 Jeu de données immobilier fictif 44
3 Caractéristiques de la nouvelle maison à prédire 49
4 Jeu de données immobilier fictif 53
5 Caractéristiques de la nouvelle maison à prédire. 54
6 Fonctionnement de la validation croisée à 12 groupes 56
7 Valeurs optimales du paramètre λ 67

I. Introduction et objectifs

Ce projet a pour but de prédire les prix des logements de la ville de Ames entre 2006 et 2010 puis d'identifier et d'analyser les caractéristiques qui influencent le plus le prix des logements. Le jeu de données utilisé est le jeu de donnée Ames Housing Dataset constitué par Dean De Cock.

En pratique, nous disposons de n observations cibles influencées par p variables explicatives. La variable cible que l'on souhaite prédire est le prix de vente des maisons (SalePrice).

Les variables qui expliquent ce prix sont par exemple :

- La Surface habitable(GrLivArea),
- la taille du terrain(LotArea),
- Qualité générale de la maison(OverallQual),
- Année de construction(YearBuilt),
- Quartier(Neighborhoood),
- Surface du garage(GarageArea),
- Nombre de voitures que peut contenir le garage(GarageCarsw),
- etc.

Pour prédire le prix à partir de l'information renvoyée par ces variables, nous utiliserons plusieurs algorithmes différents de machine learning.

II. Chargement des données

Les données sont au format csv, nous utilisons pandas pour les lire en python. Nous en affichons un extrait ci-après :

Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	...	PoolArea	PoolQC	Fence	MiscFeature	MiscVal	....	SalePrice
1	60	RL	65.0	8450	Pave	NaN	Reg	...	0	NaN	NaN	NaN	0	...	208500
2	20	RL	80.0	9600	Pave	NaN	Reg	...	0	NaN	NaN	NaN	0	...	181500
3	60	RL	68.0	11250	Pave	NaN	IR1	...	0	NaN	NaN	NaN	0	...	223500
4	70	RL	60.0	9550	Pave	NaN	IR1	...	0	NaN	NaN	NaN	0	...	140000
5	60	RL	84.0	14260	Pave	NaN	IR1	...	0	NaN	NaN	NaN	0	...	250000
6	50	RL	85.0	14115	Pave	NaN	IR1	...	0	NaN	MnPrv	Shed	700	...	143000
7	20	RL	75.0	10084	Pave	NaN	Reg	...	0	NaN	NaN	NaN	0	...	307000
8	60	RL	NaN	10382	Pave	NaN	IR1	...	0	NaN	NaN	Shed	350	...	200000
9	50	RM	51.0	6120	Pave	NaN	Reg	...	0	NaN	NaN	NaN	0	...	129900
10	190	RL	50.0	7420	Pave	NaN	Reg	...	0	NaN	NaN	NaN	0	...	118000

FIGURE 1 – Extrait des dix premières observations du jeu de données Ames Housing.

III. Familiarisation avec les données

Nous affichons ci-dessous quelques informations sur le jeu de données afin de mieux nous familiarisé avec ce dernier.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1460 entries, 0 to 1459
Data columns (total 81 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             1460 non-null   int64
 1   MSSubClass     1460 non-null   int64
 2   MSZoning       1460 non-null   object
 3   LotFrontage    1201 non-null   float64
 4   LotArea        1460 non-null   int64
 5   Street         1460 non-null   object
 6   Alley          91 non-null     object
 7   LotShape       1460 non-null   object
 8   LandContour    1460 non-null   object
 9   Utilities      1460 non-null   object
 10  LotConfig      1460 non-null   object
 11  LandSlope      1460 non-null   object
 12  Neighborhood   1460 non-null   object
 13  Condition1     1460 non-null   object
 14  Condition2     1460 non-null   object
 15  BldgType       1460 non-null   object
 16  HouseStyle     1460 non-null   object
 17  OverallQual    1460 non-null   int64
 18  OverallCond    1460 non-null   int64
 19  YearBuilt      1460 non-null   int64
 20  YearRemodAdd   1460 non-null   int64
  .      .              .            .
  .      .              .            .
  .      .              .            .
  .      .              .            .
  .      .              .            .
 78  SaleType       1460 non-null   object
 79  SaleCondition  1460 non-null   object
 80  SalePrice      1460 non-null   int64
dtypes: float64(3), int64(35), object(43)
memory usage: 924.0+ KB
```

Le jeu de données possède donc au total 1460 observations et 81 variables. Parmi ces 81 variables 79 sont des variables explicatives. En effet, la variable SalePrice est le prix de vente des maisons que l'on souhaite prédire, c'est donc notre variable cible. Et la variable Id n'a aucun intérêt pour prédire le prix.
Nous avons 3 types de données : int64 (entier), float64 (nombre réel), object (texte).
Plusieurs variables contiennent des valeurs manquantes : c'est le cas par exemple de LotFrontage, Alley, MasVnrType, MasVnrArea, etc...

Nous affichons ci-après un petit résumer concernant la variable cible SalePrice :

```
Taille du jeu de données : (1460, 81)

Fourchette des prix des logements : $34 900 - $755 000

Moyenne des prix des logements : $180 921

Médiane des prix des logements : $163 000
```

On peut remarquer que :

- La maison la moins chère vaut 34 000 dollars tandis que la maison la plus chère vaut 755 000 dollars. Donc le dataset semble contenir des maisons modestes mais aussi des maisons très haut de gamme. Il y a une grande variabilité des prix.
- Le prix qui coupe le dataset en deux est 163 000. Cela signifie qu'il y a autant de logements qui coûtent moins de 163 000 dollars que de logements qui coûtent plus de 163 000 dollars.
- La moyenne est supérieur à la médiane ce qui pourrait indiquer une distribution asymétrique vers la droite. C'est à dire quelques maisons très chères qui "tirent" la moyenne des prix vers le haut.

NB : En général, une distribution étalée à gauche présente la configuration mode > médiane > moyenne et l'étalement à droite l'ordre inverse.

IV. Analyse exploratoire et prétraitement des données

IV.1. Distribution de la variable cible SalePrice

Dans cette sous-section, nous analysons dans un premier temps la distribution de la variable cible, ce qui nous permettra de mieux comprendre comment les prix sont répartis.
Puis nous examinerons dans un second temps l'effet d'une transformation logarithmique sur la répartition des prix

IV.1.1. Coefficient d'asymétrie

Dans la partie III, nous avons mis en évidence le fait que la distribution des prix pouvaient présenter une asymétrie vers la droite.
Dans la suite nous confirmer cette hypothèse, en observant la représentation graphique de la distribution des prix et en considérant en meme temps un outil de mesure appeler skweness.

Le skweness ou **coefficient d'asymétrie** est une mesure de l'asymétrie de la distribution d'une variable.
Étant donné une variable aléatoire réelle X de moyenne μ et d'écart type σ, on définit son coefficient d'asymétrie comme le moment d'ordre trois de la variable centrée réduite :

$$\gamma_1 = \mathbb{E}\left[\left(\frac{X-\mu}{\sigma}\right)^3\right]$$

Lorsque :

- $\gamma_1 > 0$, cela indique une asymétrie vers la droite, c'est à dire une queue de distribution étalée vers la droite.
- $\gamma_1 < 0$, cela indique une asymétrie vers la gauche, c'est à dire une queue de distribution étalée vers la gauche.
- Une distribution symétrique a un coefficient $\gamma_1 \approx 0$.

Interprétation classique du skewness

Valeur du skew	**Interprétation**
$skew \approx 0$	Distribution symétrique
$-0.5 < skew < 0.5$	Asymétrie faible
$-1 < skew < -0.5$	Asymétrie modérée à gauche
$0.5 < skew < 1$	Asymétrie modérée à droite
$skew < -1$	Forte asymétrie à gauche
$skew > 1$	Forte asymétrie à droite

TABLE 1 – Interprétation classique du coefficient d'asymétrie (skewness).

Nous représentons ci-dessous l'histogramme de la distribution des prix des maisons ainsi que le QQ-plot associé :

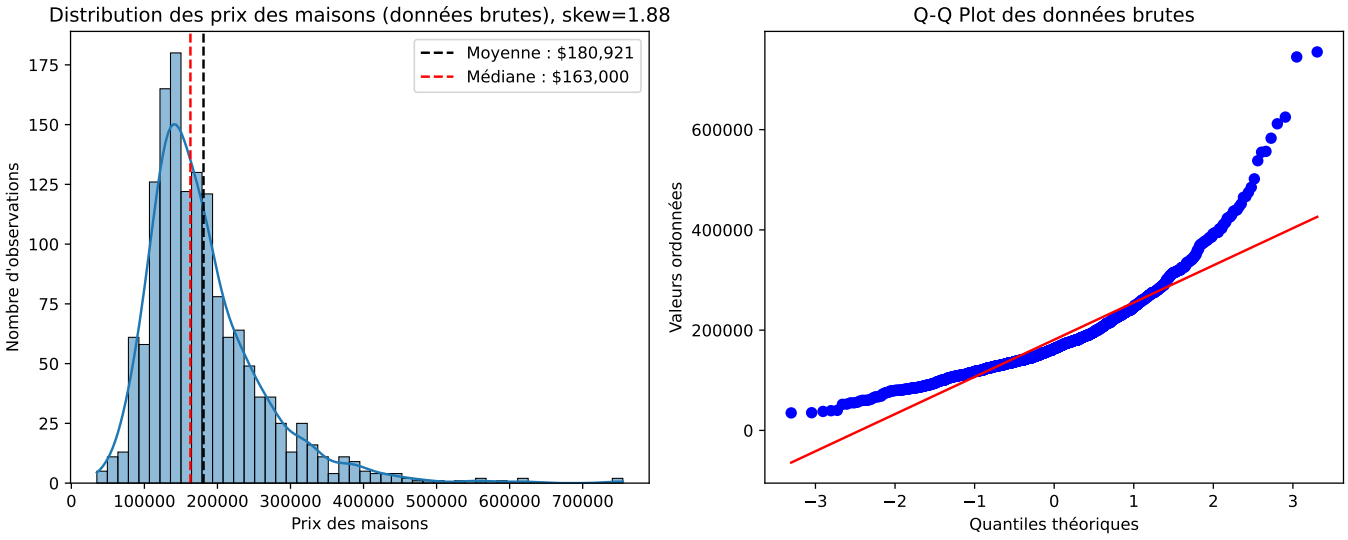


FIGURE 2 – Histogramme de la distribution des prix des maisons et QQ-plot associé.

Le skewness élevé ($skew = 1.88 > 1$), le graphique de la distribution des prix ainsi que le QQ-plot confirment notre intuition selon laquelle la distribution des prix des maisons serait fortement asymétrique vers la droite. En effet, on observe la présence de valeurs extrêmes importantes : quelques maisons très chères créent une longue queue vers la droite.

Des modèles de régression linéaire construits sur de telles données pourraient être sensibles à ces valeurs extrêmes.

Une des solutions envisagées est généralement d'effectuée une transformation sur les données. Bien que les principaux modèles étudiés dans cette étude (Random Forest, Extra Trees, Decision Tree Regressor (CART), Gradient Boosting, XGBoost) soient généralement peu sensibles à l'asymétrie des variables, nous appliquons tout de même une transformation logarithmique sur les prix des maisons. En effet, cette transformation est très bénéfique pour la régression Ridge et peut également améliorer les performances prédictives des modèles basés sur les arbres de décisions.

IV.1.2. Transformation logarithmique

Maintenant examinons brièvement ce qu'il se passe dans les faits lorsqu'on applique une transformation logarithmique sur les données. Pour cela, interessons-nous brièvement à la fonction logarithme népérien :

La fonction **logarithme népérien** :

$$\begin{array}{rcl} \ln : \mathbb{R}_+^* & \to & \mathbb{R} \\ x & \mapsto & \ln(x) \end{array}$$

est une fonction qui croit lentement. C'est à dire que même quand x devient énorme le logarithme augmente peu.
Par exemple :

$ln(10) \approx 2,3$

$ln(100) \approx 4,6$

$ln(1000) \approx 6,9$

On peut le voir plus nettement avec le graphique suivant :

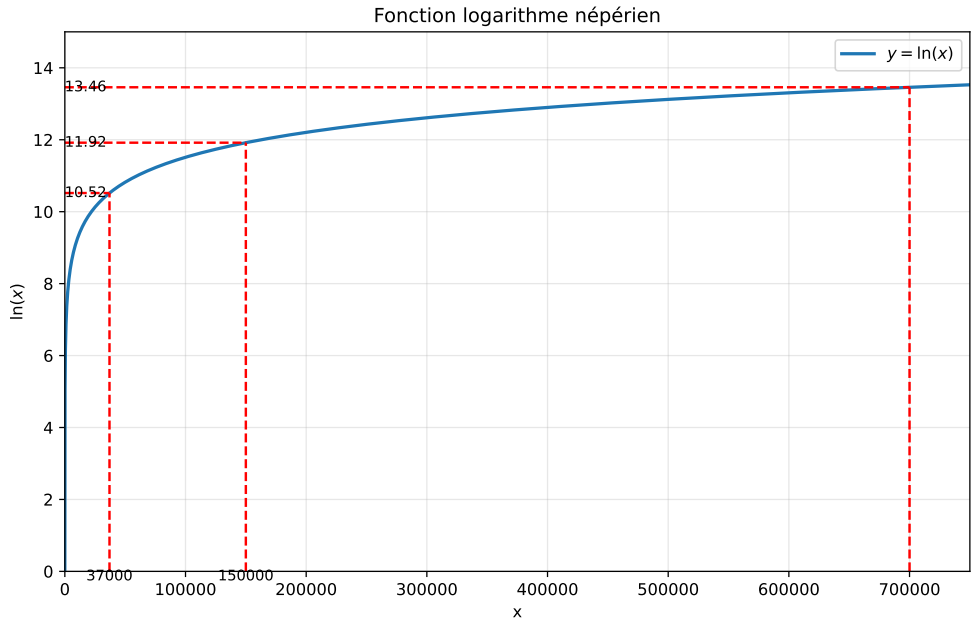


FIGURE 3 – Fonction logarithme népérien

Visuellement la courbe monte mais de plus en plus lentement. Et comme on peut le voir, les écarts deviennent extrêmement moins violent sur le logarithme.

L'idée d'utiliser le logarithme ici est donc bien évidemment qu'il va avoir pour effet de compresser les très grandes valeurs, ce qui va contribuer à réduire fortement l'asymétrie en rapprochant les valeurs extrêmes du reste des données. De façon générale, le modèle apprendra sur une cible moins dominée par les maisons de luxe ce qui aura pour effet de réduire l'impacte des maisons très chères sur des métriques comme la RMSE (ou la RMSLE).

On peut comparer avec d'autres fonctions et on voit rapidement que : $ln(x) << x^{\alpha} << \exp(x)$. Donc le logarithme croit plus lentement que toute les puissances x^{α} et énormément plus lentement que l'exponentielle. D'où le choix de cette fonction au détriment des autres fonctions.

En pratique, il est courant d'utilisé $ln(1+x)$ pour éviter les problèmes numériques lorsque les données peuvent contenir des valeurs nulles ou lorsqu'elles sont très proches de zéro. Mais dans cette étude, ce ne sera pas nécessaire car la variable cible SalePrice > 0 (voir également la plage des prix présentées ci-dessus).

D'un autre coté, la différence est minuscule pour les très grandes valeurs.
Par exemple :

$ln(35\ 000) \approx 10,4631$

$ln(1 + 35\ 000) = ln(35\ 001) \approx 12,4631$

$ln(200\ 000) \approx 12,2061$

$ln(1 + 200\ 000) = ln(200\ 001) \approx 12,2061$

Les graphiques suivants montrent les bénéfices apportées par la transformation logarithmique sur l'asymétrie de la variable cible SalePrice :

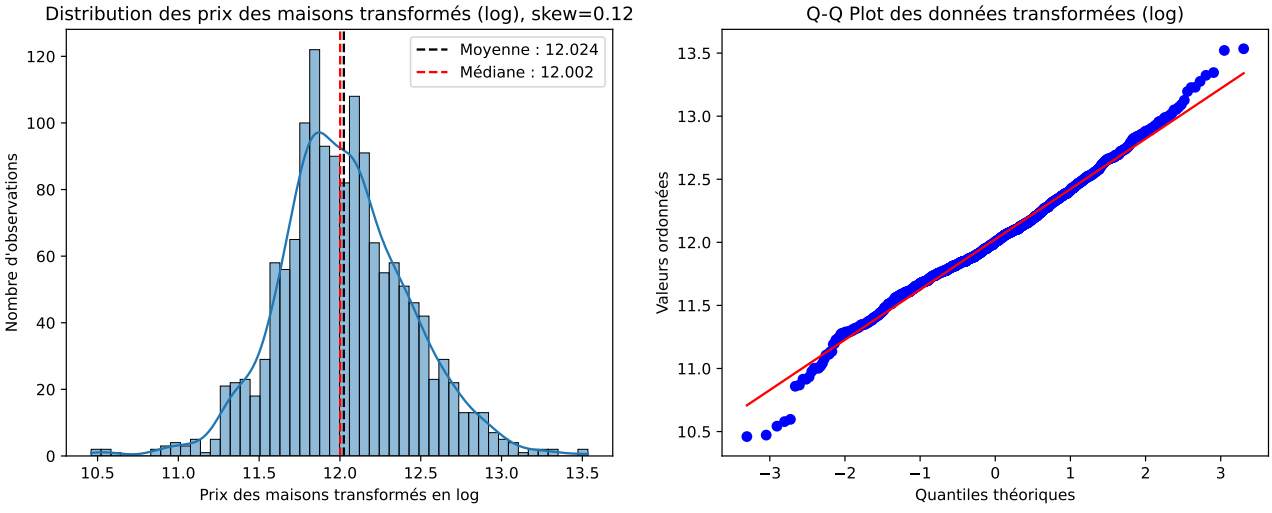


FIGURE 4 – Histogramme de la distribution des prix des maisons transformées et QQ-plot associé.

Le faible coefficient d'asymétrie ($skew = 0,12 \approx 0$), le graphique de la distribution des prix transformées ainsi que le QQ-plot montrent que la transformation logarithmique améliore significativement la symétrie de la distribution des prix, et la rapproche beaucoup plus d'une loi

normale.
L'asymétrie a fortement diminué car les valeurs extrêmes ont été "compressées" par le logarithme et ainsi rapprochées du reste des données.
Ces valeurs influences moins la distribution.

Dans la suite de cette étude, les modèles sont entraînés sur la variable cible transformée $ln(Saleprice)$ (ou $ln(1 + SalePrice)$).
Donc au lieu de prédire Saleprice, le modèle apprendra à prédire $ln(SalePrice)$ (ou $ln(1 + SalePrice)$).
Mais ce ne sont pas directement des prix, ce sont des logarithmes de prix.
Pour revenir aux vrais prix après prédictions, on distingue deux cas :

- Si on a utilisé $ln(x)$:

  $y = ln(x) \iff x = \exp(y)$
- Si on a utilisé $ln(1 + x)$:

  $y = ln(1 + x) \iff x = \exp(y) - 1$

Dans cette étude nous utilisons $ln(x)$.

Résumer de cette sous-partie :

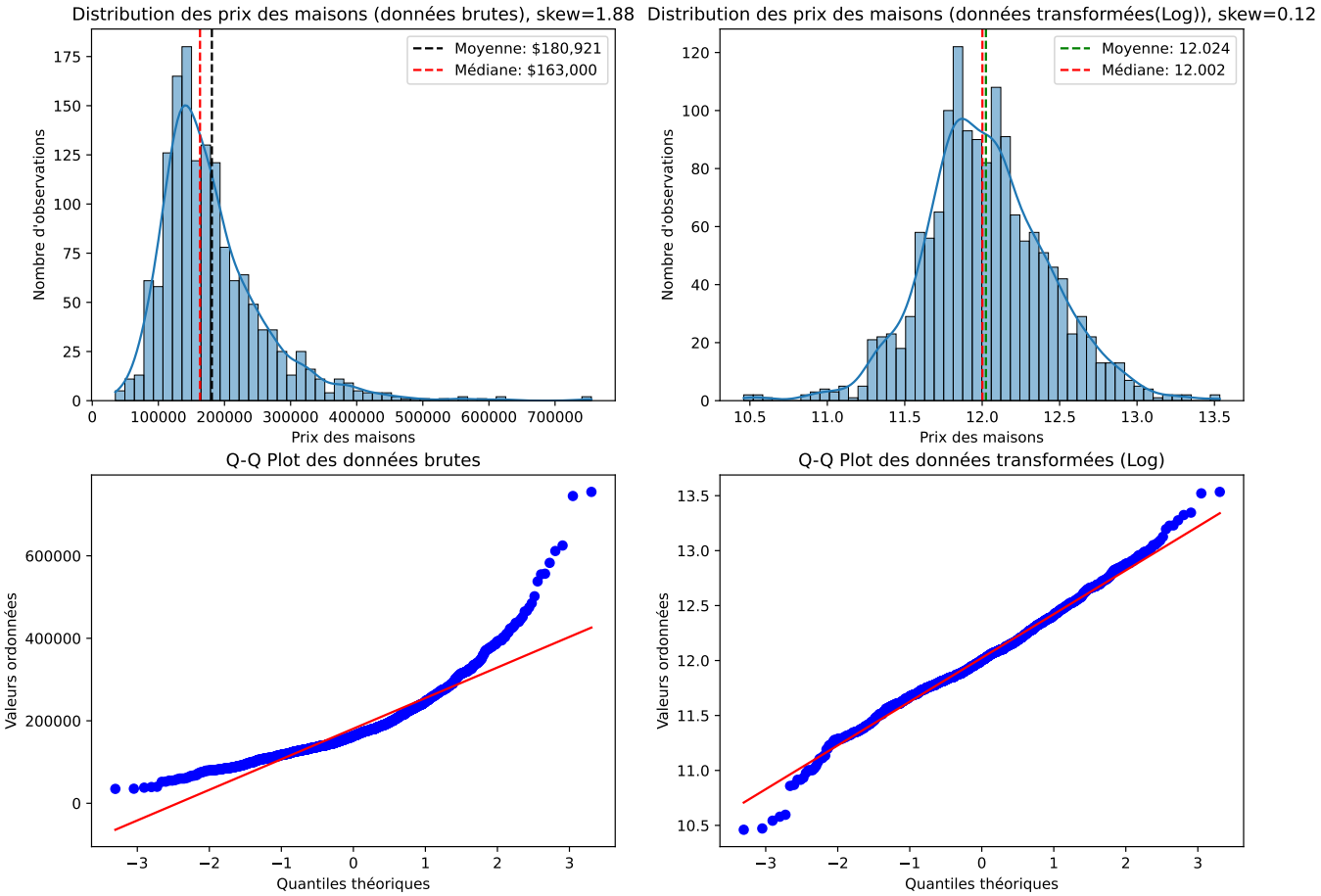


FIGURE 5 – Résumer sur la distribution de la variable cible

IV.2. Gestions des valeurs manquantes

Avant toute chose, nous nous assurons que le dataset ne contient pas de doublons :

```
Nombre d'observations doubles : 0
```

Lors de la familiarisation avec les données en partie III, nous avons remarqué et mis en évidence certaines des variables contenant des valeurs manquantes.
Nous affichons ci-dessous la liste complète de ces variables, le nombre de valeurs manquantes ainsi que la proportion et le type associés.

	Missing Count	Missing %	Data Type
PoolQC	1453	99.520548	object
MiscFeature	1406	96.301370	object
Alley	1369	93.767123	object
Fence	1179	80.753425	object
MasVnrType	872	59.726027	object
FireplaceQu	690	47.260274	object
LotFrontage	259	17.739726	float64
GarageType	81	5.547945	object
GarageYrBlt	81	5.547945	float64
GarageFinish	81	5.547945	object
GarageQual	81	5.547945	object
GarageCond	81	5.547945	object
BsmtExposure	38	2.602740	object
BsmtFinType2	38	2.602740	object
BsmtQual	37	2.534247	object
BsmtCond	37	2.534247	object
BsmtFinType1	37	2.534247	object
MasVnrArea	8	0.547945	float64
Electrical	1	0.068493	object

Dans ce dataset, les valeurs manquantes (NAN) n'ont pas toutes la même signification pour chaque variable. Pour certaines variables, les valeurs manquantes (NAN) représentent le fait que la caractéristique est manquante (non-enregistrée) pour les maisons concernées, pour d'autres variables, elles représentent plutôt le fait que les maisons ne possèdent pas la caractéristique examinée.
Par exemple, on verra que toutes les valeurs manquantes (NAN) de la variable PoolQC représente le fait que les maisons concernées ne possèdent pas de piscine et que la présence de valeurs

manquantes (NAN) dans la variable LotFrontage signifie que la mesure est manquante pour les maisons concernées.Et bien sur pour certaines variables (très peu), les valeurs manquantes (NAN) concerneront les deux cas de figure énumérés précédemment.

Dans la plupart des cas, pour statuer sur la signification définitive de NAN des maisons d'une variable, afin de décider comment les traitées, nous examinons lorsqu'elle(s) existe(nt), une(ou plusieurs) variable(s) qui lui(leurs) est(sont) directement associée(s).

IV.2.1. PoolQC

La variable catégorielle PoolQC qui décrit la qualité de la piscine possède plus de 99% (1453) de valeurs manquantes.
Une variable qui lui est directement associée est la variable PoolArea (aire de la piscine) qui elle ne possède aucunes valeurs manquantes.
Cela pourrait laisser penser que les NAN dans PoolQC ne signifient pas que la maison ne possède pas de piscine mais plutôt la qualité de la piscine est manquante (non enregistrée).
Toutefois, un examen attentif de la variable PoolArea nous montre que sur 1460 maisons seulement 7 maisons ont une aire non-nulle.

PoolArea	count
0	1453
512	1
648	1
576	1
555	1
480	1
519	1
738	1

FIGURE 6 – Valeurs prises par la variable PoolArea

On se retrouve alors dans les cas de figure suivants :

- PoolArea > 0 et PoolQC présente, représenterait les maisons avec une piscine et dont la qualité a été enregistrée.

- PoolArea= 0 et PoolQC manquante représenterait les maisons qui ne possèdent pas de piscine
- Et les maisons avec une PoolArea= 0 et PoolQC enregistrée pourraient représentées des anomalies.

Nous vérifions ci-après le nombre d'élément dans chacune des catégories précédentes :

```
Maisons où l'aire de la piscine (PoolArea) est non nulle et la qualité (PoolQC) NAN : 0
Maisons où l'aire de la piscine (PoolArea) est nulle et la qualité (PoolQC) NAN : 0
Maisons où l'aire de piscine (PoolArea) est nulle et la qualité (PoolQC) renseignée : 0
```

Toutes les maisons où la qualité de la piscine (PoolQC) est manquante (NAN) correspondent exactement aux mêmes maisons qui ont une aire de piscine (PoolArea) nulle. Or une aire de piscine (PoolArea) nulle est attribuée aux maisons sans piscine.
Donc les valeurs manquantes (NAN) de la variable PoolQC, représentent le fait que les maisons correspondantes ne possèdent pas de piscine.
Nous remplacerons donc ces valeurs manquantes (NAN) par des 'None'.

IV.2.2. MiscFeature

La variable catégorielle MiscFeature qui représente l'équipement ou l'aménagement particulier non couvert par les autres catégories possède plus de 96% (1406) de valeurs manquantes.
Cette variable a également une variable numérique associée qui est MiscVal qui elle n'a aucunes valeurs manquantes et représente la valeur monétaire associée à l'équipement.
En examinant cette variable, on peut constater qu'elle possède 1408 valeurs nulles et 52 valeurs non nulles :

```
Valeurs nulles de la variable MiscFeature : 1408
Valeurs non nulles de la variable MiscFeature : 52
```

Comme précédemment, on examine ce qui se passe dans chacun des cas de figure suivants :

```
Nombre de maisons où MiscVal est non nulle et MiscFeature est absente (NAN) : 0
Nombre de maisons où MiscVal est nulle et MiscFeature est absente (NAN) : 1406
Nombre de maisons où MiscVal est nulle et MiscFeature est renseignée : 2
```

Nous constatons que lorsque la valeur monétaire de l'équipement (MiscVal) est non nulle, le type d'équipement (MiscFeature) est toujours renseigné. Et lorsque la valeur monétaire de l'équipement est nulle, le type d'équipement est manquant (NAN) pour 1406 maisons (la totalité de NAN de la variable MiscFeature) et renseigné pour 2 maisons seulement.
Ce dernier cas de figure semble poser problème, car on a deux équipements renseignés mais possédant chacun une valeur monétaire nulle.

Regardons de quelles observations il s'agit :

Id	MiscFeature	MiscVal
874	Othr	0
1201	Shed	0

FIGURE 7 – Les deux maisons avec MiscVal nulle et MiscFeature renseigné

La maison 874 possède un équipement "Autre" et la maison 1201 possède un abri de jardin, mais aucune valeur monétaire n'a été attribuée à ces équipements.
Ce n'est probablement pas une erreur, ces équipements ont peut être bien une valeur monétaire nulle. Dans tous les cas, cela représente seulement 2 observations sur 1460. Ces valeurs resterons inchangées dans les deux variables (MiscVal et MiscFeature).

On en déduit au final que les valeurs manquantes (NAN) de la variable MiscFeature représentent le fait que les maisons concernées ne possèdent pas d'équipement particulier (MiscFeature).
Ces valeurs manquantes seront remplacées par des 'None'.

IV.2.3. MasVnrType et MasVnrArea

Les variables MasVnrType et MasVnrArea qui représentent respectivement le type de revêtement de la façade de la maison et l'aire associée à ce revêtement, ont respectivement plus de 59% (872) et moins de 1% (8) de valeurs manquantes.
En commençant par examiner ce que vaut MasVnrType pour les 8 valeurs manquantes de MasVnrArea.
On constate que les valeurs de MasVnrType sont manquantes (NAN) lorsque les valeurs de MasVnrArea le sont aussi :

Id	MasVnrType	MasVnrArea
235	NaN	NaN
530	NaN	NaN
651	NaN	NaN
937	NaN	NaN
974	NaN	NaN
978	NaN	NaN
1244	NaN	NaN
1279	NaN	NaN

FIGURE 8 – Les 8 valeurs manquantes de la variable MasVnrArea

Donc les 8 valeurs manquantes de MasVnrArea peuvent être mise à 0 et leur équivalent dans MasVnrType à 'None'.
Ensuite, on valide le fait que la variable MasVnrArea n'a donc plus de valeurs manquantes. La suite de l'analyse des valeurs manquantes restantes de la variable catégorielle MasVnrType devient alors similaire à celles des variables catégorielles précédentes.

Et en examinant attentivement la variable MasVnrArea, on constate qu'elle possède 861 valeurs nulles (sans avoir replacé les 8 NAN par 8 zéros) et 599 valeurs non nulles (sans avoir retiré les 8 NAN qui doivent être remplacé par des zéros).

```
Valeurs nulles de la variable MasVnrArea (8 zéros manquants) : 861
Valeurs non nulles de la variable MasVnrArea (8 NAN compris) : 599
```

Vérifions alors ce que vaut MasVnrType lorsque MasVnrArea est nulle ou manquant puis lorsque MasVnrArea est non-nulle et non-manquant :

```
Maisons où MasVnrArea est non nulle et non-manquante et MasVnrType est manquante (NAN) : 5
Maisons où MasVnrArea est nulle ou manquante et MasVnrType est manquante (NAN) : 867
Maisons où MasVnrArea est nulle et MasVnrType est renseigné : 2
```

On peut constater que 867 des 872 données manquantes de la variable MasVnrType seront interprétées comme étant des 'None'.
Les 5 autres données manquantes restantes correspondent à des maisons qui possèdent une aire de revêtement non nulle. Ces 5 valeurs manquantes seront remplacées en respectant les proportions de la distribution de la variable MasVnrType en ne considérant que les valeurs déjà présentes.
Nous avons aussi deux lignes où le type de revêtement est renseigné mais avec une aire de revêtement nulle :

Id	MasVnrType	MasVnrArea
689	BrkFace	0.0
1242	Stone	0.0

FIGURE 9 – Les deux maisons avec MasVnrArea nulle et MasVnrType renseigné

Puisque le type a clairement été renseigné et que plus de 500 maisons ont une aire non-nulle, on peut penser qu'il s'agit d'une erreur. On remplacera chacune de ces deux valeurs nulles par l'aire médian des maisons ayant le même type de revêtement.

IV.2.4. FireplaceQu

La variable catégorielle FireplaceQu qui représente la qualité de ou des cheminées possède plus de 47% (690) de valeurs manquantes.
Cette variable possède une variable numérique associée qui est Fireplaces. La variable numérique Fireplaces représente le nombre de cheminées de la maison ; elle ne contient aucunes valeurs manquantes (NAN). Ce qui pourrait laisser à penser que les valeurs manquantes de la variable FireplaceQu représentent les maisons pour lesquelles la qualité de ou des cheminées est manquante (non-enregistrée)
Toutefois un examen attentif de la variable Fireplaces nous montre qu'elle possède 690 valeurs nulles et 770 valeurs non nulles.

```
Valeurs nulles de la variable Fireplaces : 690

Valeurs non nulles de la variable Fireplaces : 770
```

Comme pour certaines des variables précédentes, on obtient les cas de figure suivants :

- Fireplaces > 0 et FireplaceQu renseignée, représenterait les maisons avec au moins une cheminée et dont la qualité associée a été enregistrée.
- Et Fireplaces $= 0$ et FireplaceQu manquante représenterait les maisons qui ne possèdent pas de cheminée
- Les maisons avec une Fireplaces $= 0$ et FireplaceQu enregistrée pourraient représentées des anomalies.

Nous vérifions ci-après le nombre d'élément dans chaque catégorie :

```
Nombre de maisons où Fireplaces est non nulle et FireplaceQu est manquante (NAN) : 0

Nombre de maisons où Fireplaces est nulle et FireplaceQu est manquante (NAN) : 690

Nombre de maisons où Fireplaces est nulle et FireplaceQu est renseignée : 0
```

On peut constater que lorsqu'une maison possède au moins une cheminée (Fireplaces > 0) la qualité associée est toujours enregistrée. Lorsqu'une maison ne possède aucune cheminée (Fireplaces $= 0$), la qualité est toujours manquante (NAN). Toutes les maisons dont la qualité de la cheminée (FireplaceQu) est manquante (NAN) (690) correspondent exactement aux mêmes maisons qui n'ont aucune cheminée (Fireplaces $= 0$)(690).

On en déduit que toutes les valeurs manquantes (NAN) de la variable FireplaceQu représentent le fait que les maisons concernées n'ont pas de cheminée.
Ces valeurs manquantes seront remplacées par des 'None'.

IV.2.5. LotFrontage

La variable numérique LotFrontage qui représente la longueur du terrain en bordure de rue possède plus de 17% (259) de valeurs manquantes. Contrairement aux variables précédentes,

cette variable est une variable numérique qui n'a pas une unique variable qui lui est directement associée. En effet, elle est associée aux variables LotArea, LotShape, LotConfig, qui ne possède chacune aucunes valeurs manquantes. Nous n'étudieront pas ses valeurs manquantes de la même manière qu'avec les variables précédentes.
Nous savons que les terrains d'un même quartier ont souvent des tailles similaires.
De ce fait, nous remplaceront les valeurs manquantes de la variable numérique LotFrontage par les valeurs des médianes par quartier (Neighborhood).

```
LotFrontage
Valeurs nulles : 0
Valeurs non nulles : 1460

LotArea
Valeurs nulles : 0
Valeurs non nulles : 1460
```

IV.2.6. Alley, Fence

Les variables catégorielles Alley et Fence qui représentent respectivement le type d'accès à la propriété par ruelle et la qualité de la cloture, possèdent respectivement plus de 93% (1369) et plus de 80% (1179) valeurs manquantes. Ces variables ne sont associées à aucunes autres variables. On peut regarder ci-après la répartition par modalités de chacune d'elles :

```
Fence
NaN      1179
MnPrv     157
GdPrv      59
GdWo       54
MnWw       11
Name: count, dtype: int64


Alley
NaN     1369
Grvl      50
Pave      41
Name: count, dtype: int64
```

Les valeurs manquantes (NAN) (en accord avec la description du dataset) pour chacune de ces deux variables représentent le fait que les maisons concernées ne possèdent pas la caractéristique regardée. Donc pas d'accès par ruelle et/ou pas de clôture.
Ces valeurs manquantes seront remplacées par des 'None' dans les deux variables.

IV.2.7. GarageType & GarageYrBlt & GarageFinish & GarageQual & GarageCond

Les variables GarageType, GarageYrBlt, GarageFinish, GarageQual et GarageCond qui représentent respectivement le type de garage de la maison, son année de construction, son niveau d'aménagement intérieur, sa qualité et son état général, possèdent toutes plus de 5% (81) de valeurs manquantes. Chacune de ces variables est liée à la variable GarageArea. Cette dernière ne contient aucunes valeurs manquantes.
Elle possède 81 valeurs nulles et 1379 valeurs non-nulles.

```
Valeurs nulles de la variable GarageArea : 81

Valeurs non nulles de la variable GarageArea : 1379
```

Comme avec certaines des variables précédentes, les valeurs nulles concernent très certainement les maisons qui ne possèdent pas de garage. Et comme dit précédemment, chacune de ces variables possèdent plus de 5% soit exactement 81 valeurs manquantes (NAN) chacune :

```
Nombre de NAN de la variable GarageType : 81

Nombre de NAN de la variable GarageYrBlt : 81

Nombre de NAN de la variable GarageFinish : 81

Nombre de NAN de la variable GarageQual : 81

Nombre de NAN de la variable GarageCond : 81
```

Le fait que le nombre de valeurs manquantes de chacune de ces variables est égale au nombre de valeurs nulles de la variable GarageArea, nous laisse à penser que les valeurs manquantes de chacunes de ces variables correspondent exactement aux mêmes maisons qui ne possèdent pas de garage, c'est-à-dire les maisons avec une aire de garage nulle (GarageArea = 0)
On va donc croisé les garages avec une aire nulle (GarageArea = 0) et les valeurs manquantes de chacune des autres variables afin d'infirmer ou d'approuver cette supposition.

```
Nombre de maisons où GarageArea est nulle et GarageType est manquante (NAN) : 81

Nombre de maisons où GarageArea est nulle et GarageYrBlt est manquante (NAN) : 81

Nombre de maisons où GarageArea est nulle et GarageFinish est manquante (NAN) : 81

Nombre de maisons où GarageArea est nulle et GarageQual est manquante (NAN) : 81

Nombre de maisons où GarageArea est nulle et GarageCond est manquante (NAN) : 81
```

On peut voir que toutes les valeurs manquantes de chacune de ces variables correspondent exactement aux mêmes maisons où l'aire du garage (GarageArea) est nulle.
On en déduit donc que toutes les valeurs manquantes de chacune de ces variables concernent les maisons sans garage. On les remplaçera donc par des 'None' sauf pour la variable numérique GarageYrBlt ou les valeurs manquantes seront remplacées par des 0.

IV.2.8. BsmtExposure & BsmtFinType2 & BsmtQual & BsmtCond & BsmtFinType1

Les variables catégorielles BsmtExposure, BsmtFinType2, BsmtQual,BsmtCond, BsmtFinType1 qui représentent respectivement le niveau d'exposition du sous-sol, le type de la deuxième zone aménagée du sous-sol, la qualité du sous-sol, l'état général du sous-sol et le type de la première zone aménagée du sous-sol contiennent chacune plus de 2% de valeurs manquantes, soit exactement 38, 38, 37, 37, 37 valeurs manquantes respectives. Ces variables sont toutes liées à la variable numérique TotalBsmtSF. Cette dernière représente la surface totale du sous-sol. Elle ne possède aucunes valeurs manquantes.
Elle possède 37 valeurs nulles et 1423 valeurs non nulles.

```
Valeurs nulles de la variable TotalBsmtSF : 37

Valeurs non nulles de la variable TotalBsmtSF : 1423
```

Comme avec certaines des variables précédentes, les valeurs nulles concernent très certainement les maisons qui ne possèdent pas de sous-sol. Et comme dit précédemment, chacune des variables examinées possèdent plus de 2% soit exactement 38, 38, 37, 37, 37 valeurs manquantes (NAN) :

```
Nombre de NAN de la variable BsmtExposure : 38

Nombre de NAN de la variable BsmtFinType2 : 38

Nombre de NAN de la variable BsmtQual : 37

Nombre de NAN de la variable BsmtCond : 37

Nombre de NAN de la variable BsmtFinType1 : 37
```

Le nombre de valeurs manquantes des variables BsmtQual, BsmtCond et BsmtFinType1 correspond exactement au nombre de valeurs nulles de la variable TotalBsmtSF.
Le nombre de valeurs manquantes des variables BsmtExposure et BsmtFinType2 correspond à une valeur près au nombre de valeurs nulles de la variable TotalBsmtSF.

Cela laisse à penser que les maisons possédant des valeurs manquantes pour les variables BsmtQual, BsmtCond et BsmtFinType1 correspondent exactement aux memes maisons avec une aire totale de sous-sol nulle (TotalBsmtSF $= 0$).
Et que les maisons avec des valeurs manquantes dans les variables BsmtExposure et BsmtFinType2 correspondent également aux memes maisons où l'aire total du sous-sol est nulle (TotalBsmtSF$=0$), sauf pour une seule valeur manquante dans chacune de ces deux variables qui correspondrait chacune à une maison où l'aire total du sous-sol est non nulle (TotalBsmtSF$\neq 0$).
Nous croisons ci-dessous les valeurs nulles de l'aire du sous-sol avec les valeurs manquantes de chacune des 4 variables étudiées :

```
Nombre de maisons où TotalBsmtSF est nulle et BsmtExposure est manquante (NAN) : 37
Nombre de maisons où TotalBsmtSF est nulle et BsmtFinType2 est manquante (NAN) : 37
Nombre de maisons où TotalBsmtSF est nulle et BsmtQual est manquante (NAN) : 37
Nombre de maisons où TotalBsmtSF est nulle et BsmtCond est manquante (NAN) : 37
Nombre de maisons où TotalBsmtSF est nulle et BsmtFinType1 est manquante (NAN) : 37
```

Comme sous-entendu, les maisons associées aux 37 valeurs manquantes de chacune des variables BsmtQual, BsmtCond et BsmtFinType1 correspondent exactement aux memes maisons où l'aire du sous-sol est nulle. Donc les valeurs manquantes de chacune de ces 3 variables représentent le fait que la maison ne possède pas de sous-sol.
Ces valeurs manquantes seront donc remplacées par des 'None'.

De meme, les maisons associées à 37 des valeurs manquantes des variables BsmtExposure et BsmtFinType2 correspondent aux memes maisons où l'aire du sol est nulle. Donc les 37 valeurs manquantes de chacune de ces deux variables représentent le fait que les maisons concernées ne possèdent pas de sous sol.
Ces valeurs manquantes seront donc remplacées par des 'None'.

A présent, il y a une valeur manquante dans chacune des deux variables précédentes qui correspondent à des maisons qui à priori possèdent un sous-sol (c'est à dire TotalBsmtS$\neq$0 ($>$ 0)) mais dont les valeurs de BsmtExposure et BsmtFinType2 sont manquantes.
Voyons de quelles maisons il s'agit :

Id	TotalBsmtSF	BsmtExposure
949	936	NaN

Id	TotalBsmtSF	BsmtFinType2
333	3206	NaN

FIGURE 10 – Maisons avec TotalBsmtS $>$ 0 et BsmtExposure ou BsmtFinType2 manquantes

Il s'agit des maisons 949 et 333 qui possèdent respectivement les valeurs 936 et 3206 pour les surfaces totales de leur sous-sol.
Après vérification, on constate que les modalités de BsmtExposure et BsmtFinType2 les plus présentes dans ces plages de surface totale du sous-sol (TotalBsmtSF) correspondent également aux modalités d'effectif maximale (mode) de chacune des deux variables.
On remplacera chacune de ces 2 valeurs manquantes par le mode de sa variable respective :

```
BsmtExposure
No     953
Av     221
Gd     134
Mn     114
NaN     38
Name: count, dtype: int64
```

```
BsmtFinType2
Unf    1256
Rec      54
LwQ      46
NaN      38
BLQ      33
ALQ      19
GLQ      14
Name: count, dtype: int64
```

IV.2.9. Electrical

La variable Electrical possède une seule valeur manquante. Nous remplaçerons cette valeur manquante par le mode de la variable.

```
Electrical
SBrkr    1334
FuseA      94
FuseF      27
FuseP       3
Mix         1
NaN         1
Name: count, dtype: int64
```

Nous avons terminé l'analyse de toutes les variables contenant des valeurs manquantes dans le dataset. A présent, nous appliquons toutes les décisions retenues précédent à ces variables. Une fois le traitement terminé, le dataset ne contient plus aucunes valeurs manquantes :

```
Nombre de valeurs manquantes restantes dans le dataset : 0
```

IV.3. Corrélations

Le coefficient de corrélation de Pearson est une mesure qui quantifie la force et le sens de la relation linéaire entre deux variables quantitatives.

Il varie entre -1 et 1.

Plus sa valeur est proche de zéro, plus la relation linéaire entre les variables est faible.
Lorsque sa valeur est positive, cela signifie que les deux variables évoluent dans le même sens.
Lorsque sa valeur est négative , cela signifie que les deux variables évoluent dans des sens opposées. Les valeurs 1 et -1 représentent chacune des corrélations "parfaites", positive et négative respectivement.
Nous utilisons le coefficient de corrélation de Pearson afin d'identifié les variables les plus fortement correlées avec la variable cible SalePrice.

Soit N le nombre total de maisons et M le nombre de variables. Soit $(i, j) \in [1, M] \times [1, M]$ et le $s \in [1, N]$,

$$r(X_i, X_j) = \frac{cov(X_i, X_j)}{\sigma(X_i) \times \sigma(X_j)} = \frac{cov(X_i, X_j)}{\sqrt{S(X_i)} \times \sqrt{S(X_j)}} = \frac{\sum_{s=1}^{N}[X_i(s) - \bar{X}_i] \times [X_j(t) - \bar{X}_j]}{\sqrt{\sum_{s=1}^{N}[X_i(s) - \bar{X}_i]^2} \times \sqrt{\sum_{s=1}^{N}[X_j(s) - \bar{X}_j]^2}}$$

avec

$$cov(X_i, X_j) = \sum_{s=1}^{N} \frac{[X_i(s) - \bar{X}_i] \times [X_j(s) - \bar{X}_j]}{N} \ , \ S(X_i) = \frac{1}{N} \times \sum_{s=1}^{N}[X_i(s) - \bar{X}_i]^2$$

et

$$r(X_i, X_i) = 1$$

Nous visualisons ci-dessous les 15 premières variables les plus corrélées avec la variable cible SalePrice.

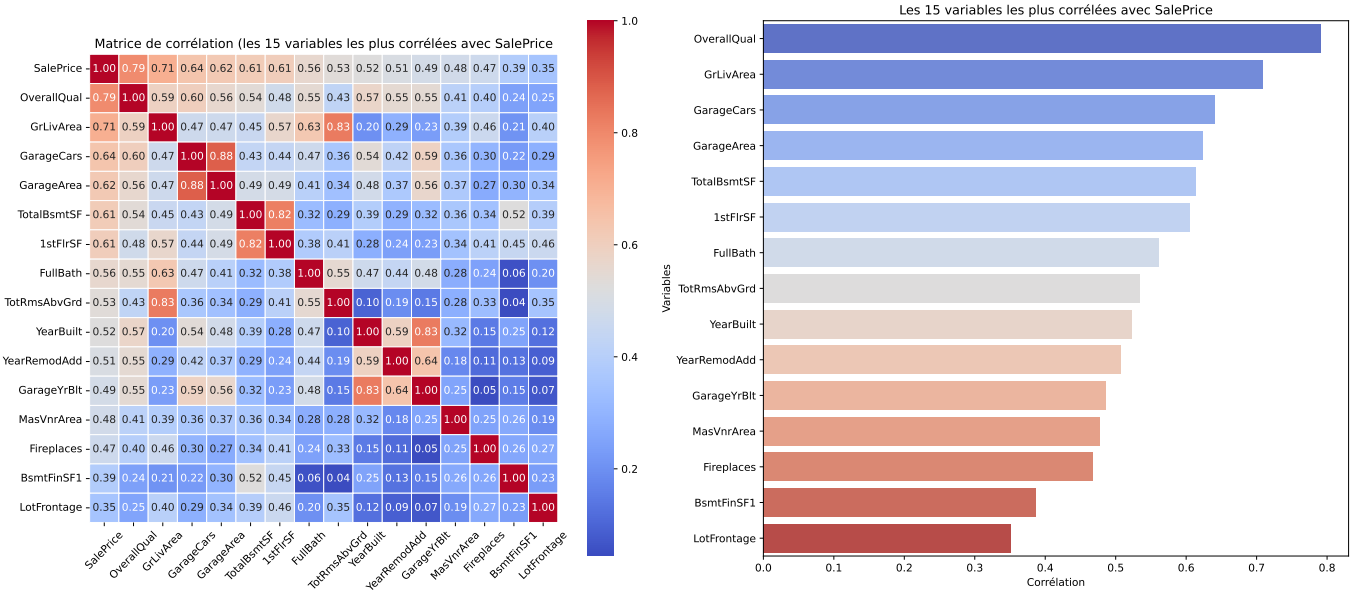


FIGURE 11 – Top 15 des variables les plus corrélées avec SalePrice

On peut constater que les variables :

- OverallQual : qualité globale des matériaux et des finitions de la maisons
- GrLivArea : surface habitable du logement au dessus du sous-sol
- GarageCars : capacité du garage en nombre de voitures
- GarageArea : surface du garage
- TotalBsmtSF : surface totale du sous-sol

- 1stFlrSF : surface du rez-de-chaussé
- FullBath : nombre de salle de bain complètes situées au-dessus du sous-sol.
- TotRmsAbvGrd : nombre total de pièces au-dessus du sous-sol
- YearBuilt : date de construction de la maison
- YearRemodAdd : date des dernières rénovation de la maison

sont celles qui sont les plus corrélées ($r > 0.5$) avec la variable cible SalePrice. Ces variables sont susceptibles d'être les plus utiles pour prédire le prix des maisons.

IV.4. Gestion des valeurs aberrantes

IV.4.1. Valeurs aberrantes des variables numériques quantitatives fortement corrélées avec SalePrice

Commençons par examiner les distributions des variables numériques quantitatives les plus fortement corrélées avec la variable cible SalePrice.

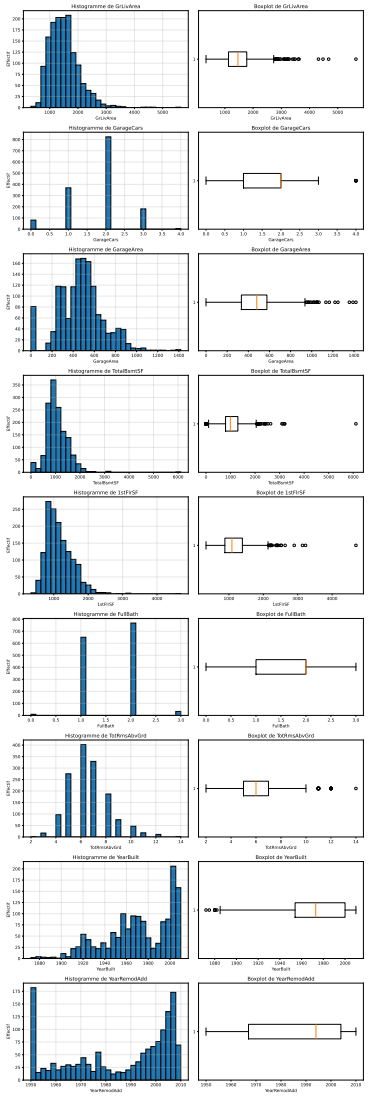


FIGURE 12 – Distributions des variables les plus fortement corrélées (>0.5) avec SalePrice

Parmi les variables examinées précédemment, celles qui semblent contenir des valeurs abérrantes sont GarageArea TotalBsmtSF, 1stFlrSF et GrLivArea.
Les nuages de points suivants qui montrent les relations entre chacune de ces variables et la variable cible SalePrice confirment cela avec la présence de certaines observations atypiques qui ne suivent pas les tendances générales.

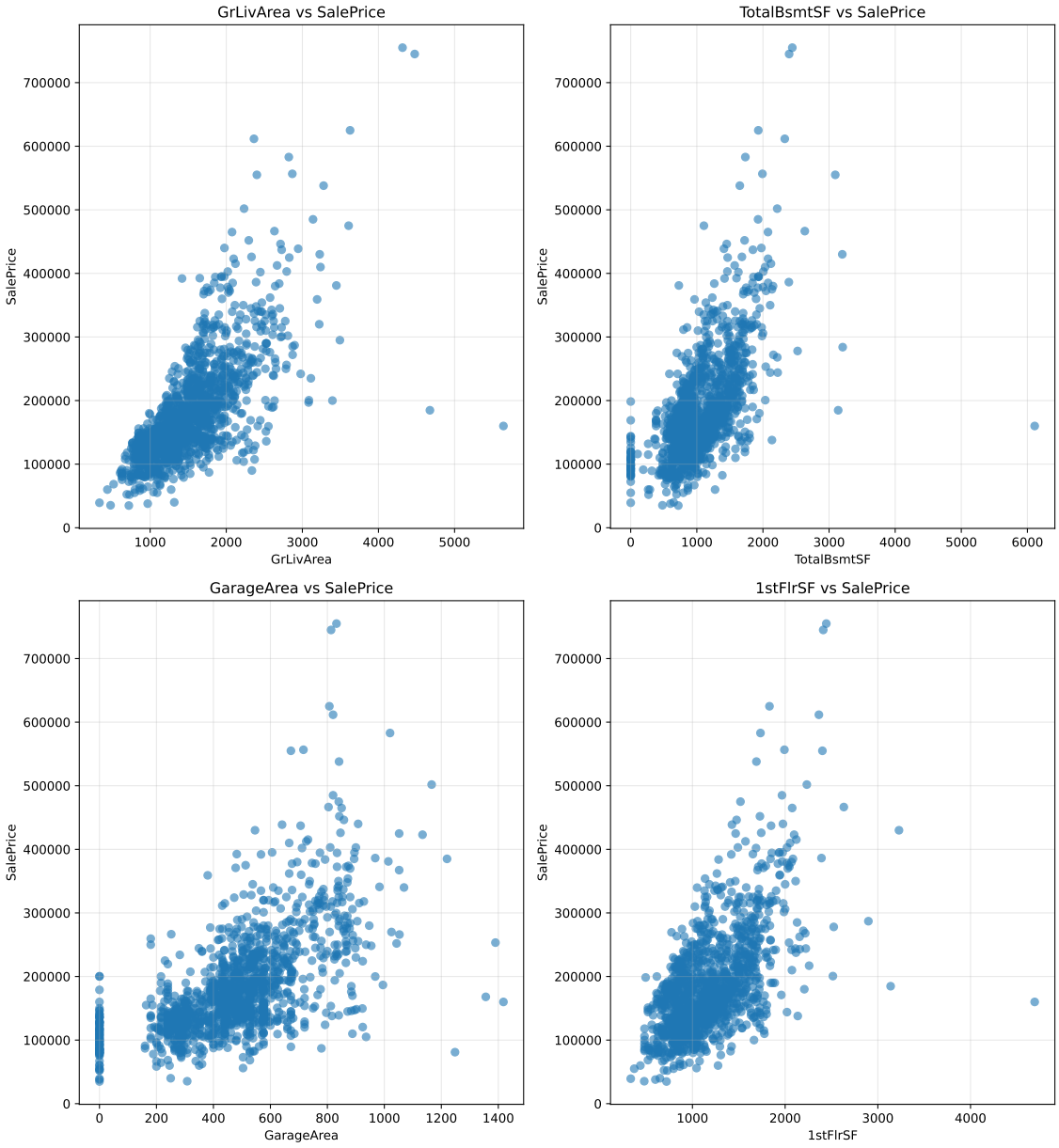


FIGURE 13 – Nuages de points de GarageArea TotalBsmtSF, 1stFlrSF et GrLivArea avec SalePrice

En effet, on peut observer deux maisons avec des prix relativement bas mais une surface habitable au dessus du sous-sol élevée (GrLivArea > 4500).
Une seule maison avec un prix relativement bas et une surface de premier étage très elevée (1stFlrSF > 4000).
Une seule maison avec un prix relativement bas et une surface totale du sous-sol très élevée (TotalBsmtSF > 5000).

IV.4.2. Valeurs aberrantes des autres variables quantitatives

A présent, considérons les autres variables numériques quantitatives du dataset.
Le dataset contient plusieurs variables numériques (int64 et float64). Parmi celles-ci, certaines sont des catégories codées par des nombres, comme par exemple la variable MSSubClass.
Donc int64 ou float64 ne signifie pas forcément que la variable est quantitative. Il va falloir repérer parmi les variables numériques (et avec l'aide de la description du dataset) celles qui sont réellement numériques quantitatives.
Pour cela, commençons par mettre en évidence toutes les variables numériques (int et float) du dataset :

```
Index(['Id', 'MSSubClass', 'LotFrontage', 'LotArea', 'OverallQual',
       'OverallCond', 'YearBuilt', 'YearRemodAdd', 'MasVnrArea', 'BsmtFinSF1',
       'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF', '1stFlrSF', '2ndFlrSF',
       'LowQualFinSF', 'GrLivArea', 'BsmtFullBath', 'BsmtHalfBath', 'FullBath',
       'HalfBath', 'BedroomAbvGr', 'KitchenAbvGr', 'TotRmsAbvGrd',
       'Fireplaces', 'GarageYrBlt', 'GarageCars', 'GarageArea', 'WoodDeckSF',
       'OpenPorchSF', 'EnclosedPorch', '3SsnPorch', 'ScreenPorch', 'PoolArea',
       'MiscVal', 'MoSold', 'YrSold', 'SalePrice'],
      dtype='object')
Nombre de variables numériques : 38
```

Parmi les variables numériques précédente, celles qui sont réellement des variables numériques quantitatives (continues ou discrètes) sont :
LotFrontge, LotArea, MasVnrArea, BsmtFinSF1, BsmtFinSF2, BsmtUnfSF, TotalBsmtSF, stFlrSF, 2ndFlrSF, YearBuilt, YearRemodAdd, YrSold, LowQQualFinSF, GrLivArea, GarageArea, WoodDeckSF, OpenPorchSF, EnclosedPorch, 3SsnPorch, ScreenPorch, PoolArea, MiscVal, GarageYrBlt, BedroomAbvGr, Kitchenabv, TotRmsAbvGrd, Fireplaces, GarageCars, BsmtFullBath, BsmtHalfBath, FullBath, HalfBath.
Parmi celles-ci, on peut retirer celles déjà examinées précédemment, il nous reste alors :
LotFrontge, LotArea, MasVnrArea, BsmtFinSF1, BsmtFinSF2, BsmtUnfSF, 2ndFlrSF, YrSold, LowQQualFinSF, woodDeckSF, OpenPorchSF, EnclosedPorch, 3SsnPorch, ScreenPorch, PoolArea, MiscVal, GarageYrBlt, BedroomabvGr, Kitchenabv, Fireplaces, BsmtFullBath, BsmtHalfBath, HalfBath.
Nous avons donc au total 23 variables quantitatives continue ou discrète.
Examinons les distributions de chacune de ces variables :

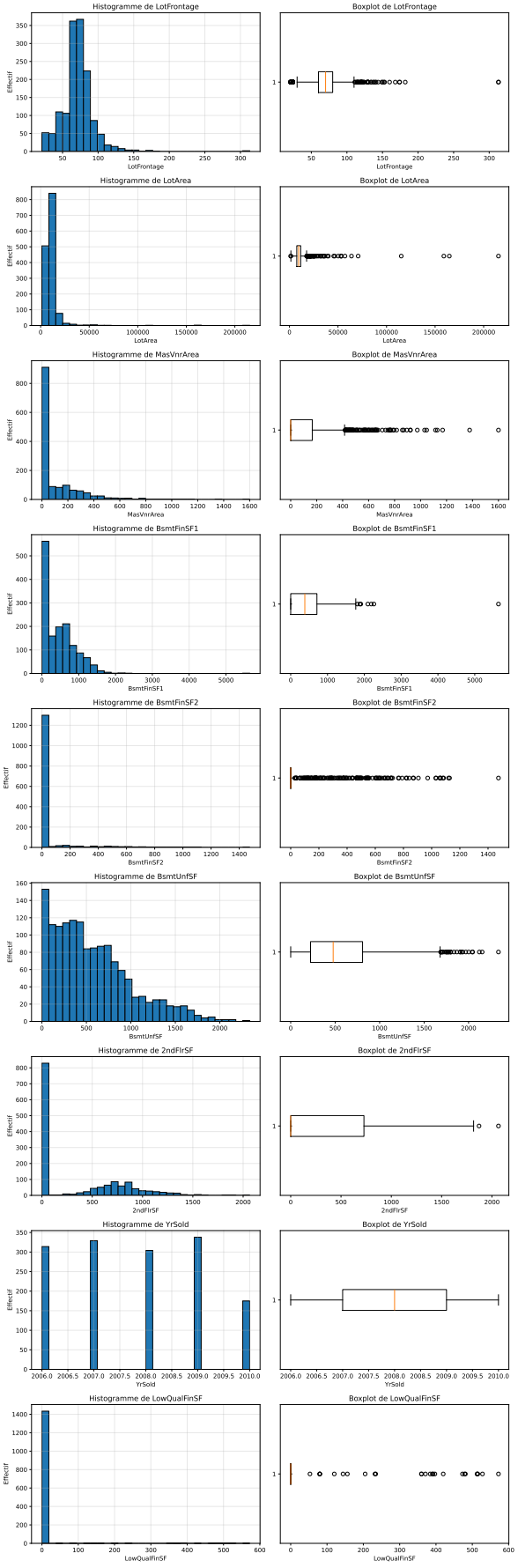


FIGURE 14 – Distribution des variables quantitatives 1

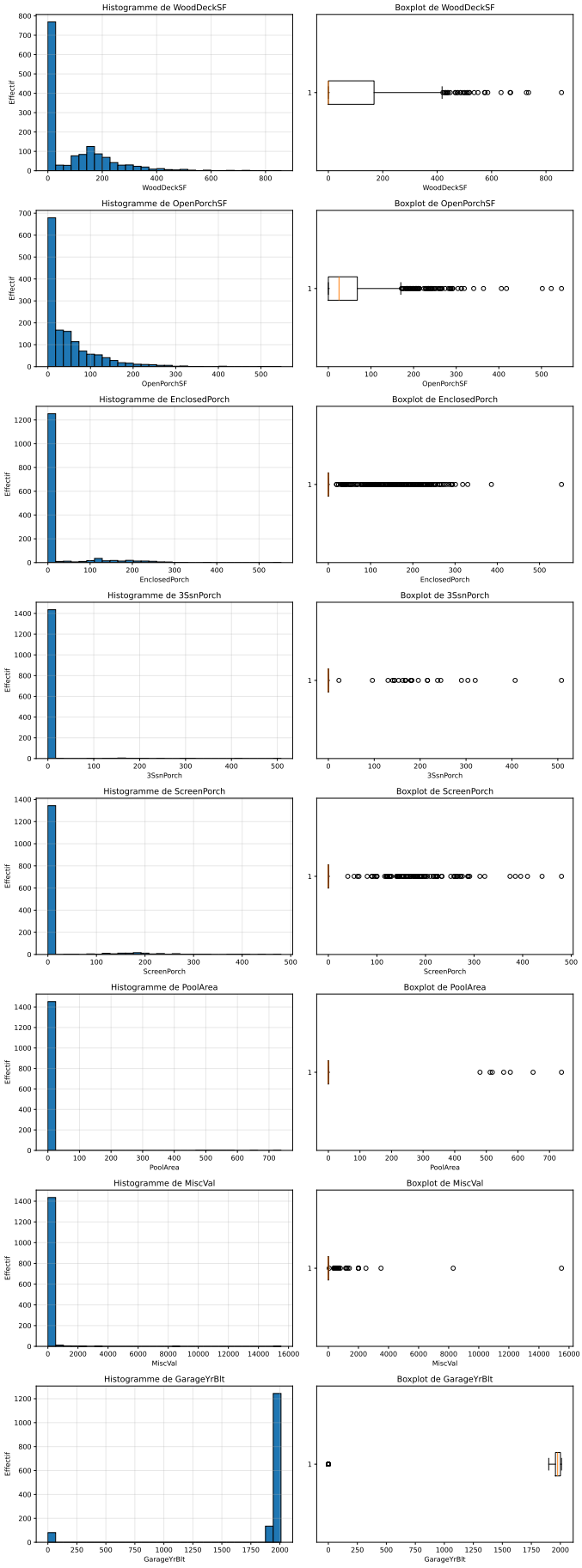


FIGURE 15 – Distribution des variables quantitatives 2

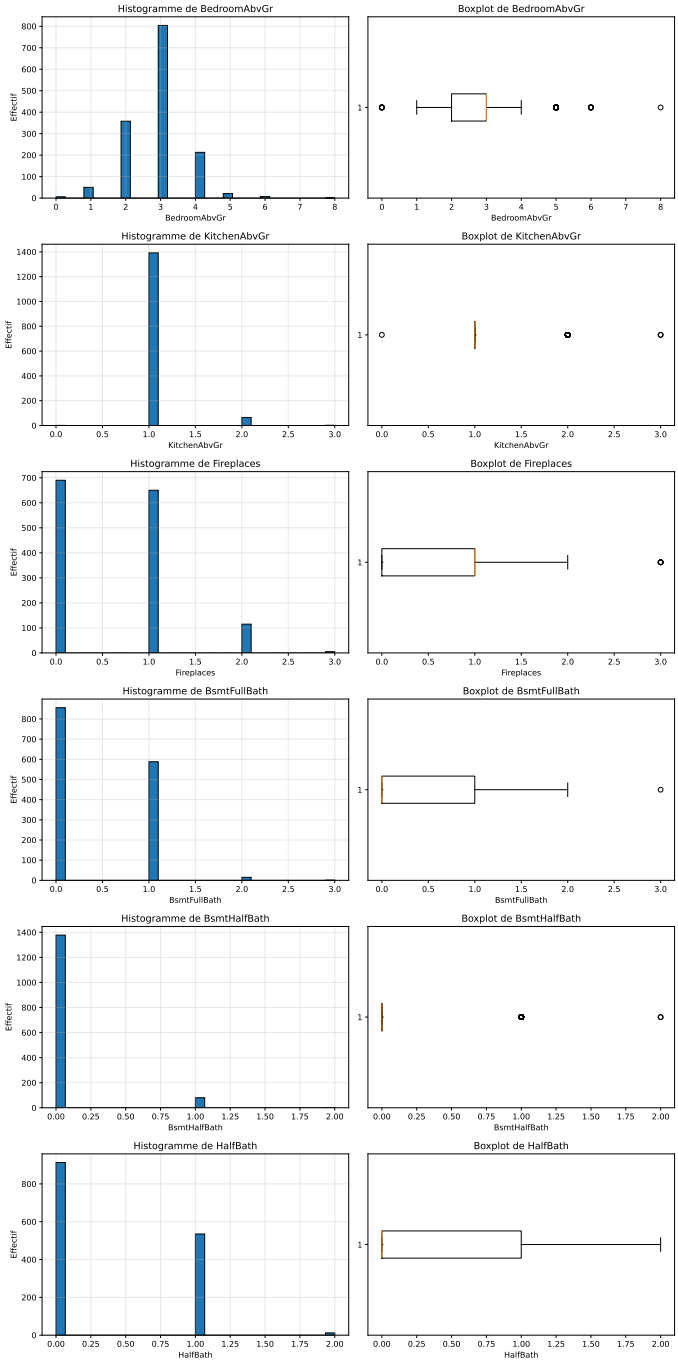


FIGURE 16 – Distribution des variables quantitatives 3

Les visualisations précédentes suggèrent que, les variables qui semblent contenir des valeurs aberrantes sont :

- LotFrontage (une valeur potentiellement aberrante) > 300
- LotArea (plus d'une valeur potentiellement aberrante) $> 100\ 000$
- BsmtFinSF1 (une valeur potentiellement aberrante) > 5000
- EnclosedPorch (une valeur potentiellement aberrante) > 500
- MiscVal (une valeur potentiellement aberrante) > 15000
- GarageYrBlt (plus d'une valeur potentiellement aberrante) < 250. Dans le cas spécifique de la variable GarageYrBlt, on se rappelle que les maisons avec une année de construction du

garage valant zéro, sont des maisons qui n'ont simplement pas de garage. Donc la valeur zéro n'est pas considérée comme aberrante pour cette variable. Idem pour les variables BsmtFinSF1, EnclosedPorch et MiscVal.

Les nuages de points suivant montrent les relations entre chacune des variables citées ci-dessus et la variable cible SalePrice. On peut constater la présence de certaines observations atypiques qui ne suivent pas les tendances générales :

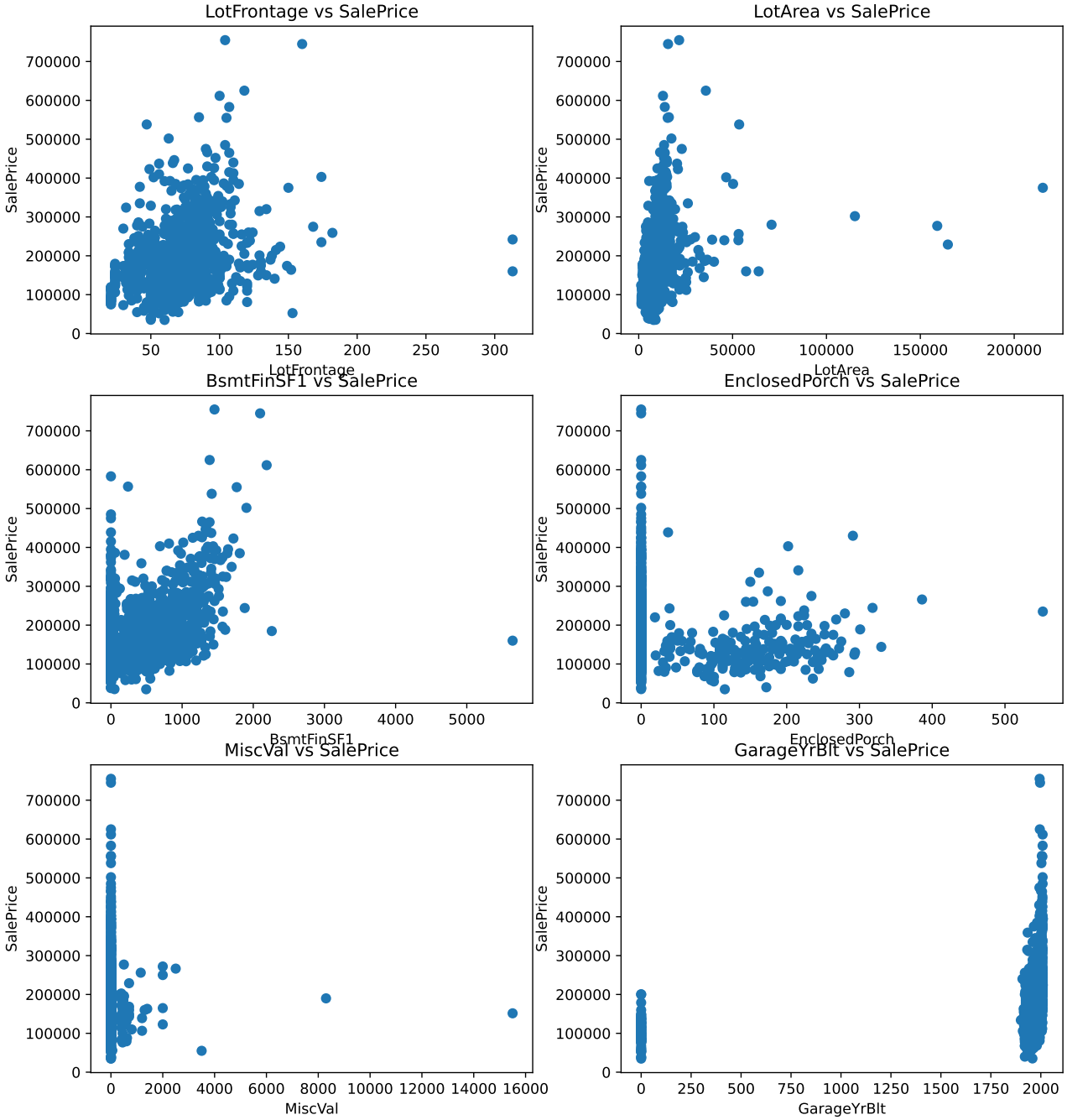


FIGURE 17 – Nuages de points de LotFrontage, LotArea, BsmtFinSF1, EnclosedPorch, MiscVal et GarageYrBlt avec SalePrice

En effet, on peut observer :

- Deux maisons avec des prix relativement bas mais une longueur de façade élevée (LotFron-

tage > 300).

- Quatre maisons avec un prix relativement bas et une surface total du terrain elevée (LotArea > 100 000)
- Une seule maison avec un prix relativement bas et une surface de la première zone aménagée du sous-sol très élevée (BsmtFinSF1 > 5000).
- Une seule maison avec un prix relativement bas et une surface d'un porche fermé élevée (EnclosedPorch > 500).
- Trois maisons avec un prix relativement bas et une valeur monétaire d' équipements divers élevée (MiscVal > 3000).

Regardons de plus prêt de quelles maisons il s'agit et comment sont leurs autres caractéristiques afin de déterminer s'il s'agit de maisons réelles mais juste avec une caractéristique exceptionnelle ou d'erreurs de saisie par exemple :

GrLivArea > 4500

Id	SalePrice	Neighborhood	OverallQual	OverallCond	YearBuilt	GrLivArea	LotFrontage	LotArea	TotalBsmtSF	MiscVal	MiscFeature
524	184750	Edwards	10	5	2007	4676	130.000000	40094	3138	0	None
1299	160000	Edwards	10	5	2008	5642	313.000000	63887	6110	0	None

1stFlrSF > 4000

Id	SalePrice	Neighborhood	OverallQual	OverallCond	YearBuilt	GrLivArea	LotFrontage	LotArea	TotalBsmtSF	MiscVal	MiscFeature
1299	160000	Edwards	10	5	2008	5642	313.000000	63887	6110	0	None

TotalBsmtSF > 5000

Id	SalePrice	Neighborhood	OverallQual	OverallCond	YearBuilt	GrLivArea	LotFrontage	LotArea	TotalBsmtSF	MiscVal	MiscFeature
1299	160000	Edwards	10	5	2008	5642	313.000000	63887	6110	0	None

LotFrontage > 300

Id	SalePrice	Neighborhood	OverallQual	OverallCond	YearBuilt	GrLivArea	LotFrontage	LotArea	TotalBsmtSF	MiscVal	MiscFeature
935	242000	NAmes	7	7	1960	2069	313.000000	27650	585	0	None
1299	160000	Edwards	10	5	2008	5642	313.000000	63887	6110	0	None

LotArea > 100000

Id	SalePrice	Neighborhood	OverallQual	OverallCond	YearBuilt	GrLivArea	LotFrontage	LotArea	TotalBsmtSF	MiscVal	MiscFeature
250	277000	ClearCr	6	7	1958	2144	80.000000	159000	1444	500	Shed
314	375000	Timber	7	5	1965	2036	150.000000	215245	2136	0	None
336	228950	Timber	5	6	1965	1786	85.000000	164660	1499	700	Shed
707	302000	ClearCr	7	5	1971	1824	80.000000	115149	1643	0	None

BsmtFinSF1 > 5000

Id	SalePrice	Neighborhood	OverallQual	OverallCond	YearBuilt	GrLivArea	LotFrontage	LotArea	TotalBsmtSF	MiscVal	MiscFeature
1299	160000	Edwards	10	5	2008	5642	313.000000	63887	6110	0	None

EnclosedPorch > 500

Id	SalePrice	Neighborhood	OverallQual	OverallCond	YearBuilt	GrLivArea	LotFrontage	LotArea	TotalBsmtSF	MiscVal	MiscFeature
198	235000	NAmes	8	4	1918	3112	174.000000	25419	1360	0	None

MiscVal > 3000

Id	SalePrice	Neighborhood	OverallQual	OverallCond	YearBuilt	GrLivArea	LotFrontage	LotArea	TotalBsmtSF	MiscVal	MiscFeature
347	151500	NAmes	6	8	1960	958	73.000000	12772	958	15500	Gar2
706	55000	IDOTRR	4	5	1930	1092	70.000000	5600	0	3500	Othr
1231	190000	Sawyer	5	5	1977	2620	71.000000	18890	1361	8300	Gar2

FIGURE 18 – Autres caractéristiques des maisons avec des potentielles valeurs aberrantes

On constate que la maison 1299 est très suspecte. En effet, elle a un prix très faible (160000) pour :

- une OverallQual maximale (10),
- une OverallCond maximale (5),
- une GrLivArea très élevé (> 4500),
- une LotFrontage très élevée (> 300),
- une YearBuilt récente (2008),
- une TotalBsmtSF très élevée (> 6000).

Toutes ces caractéristiques devraient conduire à un prix très élevé, mais ce n'est pas le cas. Il s'agit très certainement d'une anomalie. Cette maison sera supprimée.
Idem pour la maison 524 qui a des caractéristiques élevées pour un prix très faible par rapport aux autres maisons ayant des caractéristiques proches. Elle sera également supprimée.

Les 4 maisons avec LotArea $> 100\,000$ ont des prix environ entre 229000 et 375000. Ce ne sont pas forcément des anomalies. Elles ont simplement un très grand terrain. On les conservera.
La seule maison avec EnclosedPorch > 500 a un prix de 235 000 et une surface totale du terrain assez élevée. Rien ne semble incohérent. Elle sera conservée.
La maison 935 est la seconde maison avec un LotFrontage > 300. L'ensemble de ses caractéristiques semblent cohérentes au vu de son prix. Elle a juste donc une longueur de façade élevée. Cette maison sera conservée.
Pour les 3 maisons avec MiscVal > 3000. Le type d'équipement associé est : Gar2 (deuxième garage) et Oth (autre). Les autres caractéristiques associées à ces 3 maisons sont correctes par rapport au prix. Ce sont uniquement les équipement de ces maisons qui ont un prix élevé. Les 3 maisons seront conservées.

```
Maisons supprimées : [524, 1299]

Nouvelle taille du dataset : (1458, 81)
```

IV.5. Gestion des variables catégorielles (Encodage)

La plupart des modèles de machine learning (Random Forest, Gradient Boosting, XGBoost, ...) manipulent des données numériques. Ils ne savent pas traiter directement du texte (chaines de caractères). Il faut donc transformer les catégories en nombres. C'est ce qu'on appelle l'encodage. Dans cette sous-section nous effectuons deux types d'encodage différent. L'ordinal encoding et le One-Hot Encoding.

IV.5.1. Ordinal Encoding

On l'utilise lorsque les catégories d'une variable ont un ordre naturel. Par exemple, plusieurs variables du dataset qui enregistrent la qualité possèdent les catégories :

- Po (Poor)
- Fa (Fair)

- TA (Typical)
- Gd (Good)
- Ex (Excellent)

Il est clair qu'il existe bien un ordre naturel ici : **Poor** < **Fair** < **Typical** < **Good** < **Excellent**

On peut donc coder :

- "Po" : 1,
- "Fa" : 2,
- "TA" : 3,
- "Gd" : 4,
- "Ex" : 5

De cette façon on conserve l'information que Good est meilleur que Typical, par exemple.

IV.5.2. One-Hot Encoding

On l'utilise pour les variables sans ordre.

Prenons une variable comme **Neighborhood** dont les modalités sont :

- Edwards
- Timber
- NAmes
- ClearCr
- etc

Il n'y a aucun ordre entre ces quartiers.

Si on fait :

- Edwards = 1
- Timber = 2
- NAmes = 3
- ClearCr = 4
- etc

le modèle pourrait croire que : ClearCr > NAmes > Timber > Edwards > etc, alors que cela n'a absolument aucun sens.

On créer donc une colonne par quartier et on obtient une variable binaire pour chaque quartier. Ce qui peut donner par exemple pour une certaine maison :

Edwards	**Timber**	**NAmes**	**ClearCr**	...
1	0	0	0	...
⋮	⋮	⋮	⋮	⋮ ...

puis pour une autre maison :

Edwards	**Timber**	**NAmes**	**ClearCr**	...
0	1	0	0	...
⋮	⋮	⋮	⋮	⋮ ...

etc.

On remarque au passage que le One-Hot Encoding augmente le nombre de variable du dataset. Par exemple une seule variable comme Neighborhood qui possède 25 modalités est remplacée par 25 variables binaires.

IV.5.3. Mise en oeuvre

Le dataset possède 43 variables renseignée comme catégorielle (type objet) :

```
Nombre de variables catégorielles : 43
```

Celles sur lesquelles il est possible d'effectué un ordinal encoding sont :

- ExterQual
- ExterCond
- PoolQC
- PavedDrive
- BsmtQual
- BsmtCond
- KitchenQual
- Utilities
- GarageQual
- GarageCond
- FireplaceQu
- BsmtExposure
- BsmtFinType1
- BsmtFinType2
- HeatingQC
- Fence
- Functional
- LandSlope
- LotShape
- GarageFinish

Et sur le reste nous effectuons un One-Hot Encoding.

```
20 variables ont été encodées ordinalement
```

La variable MSSubClass est codée avec des nombres mais c'est en réalité une variable catégorielle. Nous la convertissons donc en variable catégorielle pour ensuite faire du One-Hot Encoding dessus également.

```
Type de MSSubClass : object

Nombre de variables catégorielles restantes : 24

One-Hot Encoding terminé

Nombre total de variables : 240

Nouvelle dimension du dataset : (1458, 240)
```

Bien entendu, on se retrouve avec beaucoup plus de variables à la fin, car comme expliquer ci-dessus, le One-Hot Encoding augmente le nombre total de variable du dataset.

Nous affichons ci-dessous un léger aperçu du One-hot-Encoding sur la variable Neighborhood par exemple :

Neighborhood_BrkSide	Neighborhood_ClearCr	Neighborhood_CollgCr	Neighborhood_Crawfor	Neighborhood_Edwards	Neighborhood_Gilbert	Neighborhood_IDOTRR
0	0	1	0	0	0	0
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	0	0	1	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
1	0	0	0	0	0	0

FIGURE 19 – Léger aperçu du One-hot-Encoding sur la variable Neighborhood

V. Entrainement des modèles (Random Forest Regressor, Extra Trees Regressor, Decision Tree Regressor (CART), Gradient Boosting Regressor, Extreme Gradient Boosting (XGBoost), Ridge Regression) et évaluation sans creation de nouvelles variables et correction d'asymétrie.

Dans cette partie, nous entraînons et évaluons plusieurs modèles de machine learning sur le jeu de données après le traitement des valeurs manquantes, des valeurs aberrantes et l'encodage des variables catégorielles, mais sans appliquer de correction de l'asymétrie sur les variables numériques (hormi SalePrice) ni créer de nouvelles variable.
Cette première expérimentation nous permettra de comparer les performances de prédictions des modèles avec et sans applications de ces deux techniques supplémentaires de prétraitement.

Avant d'entrainé les modèles sur les données, on supprime la colonne Id qui n'a pas d'utilité dans l'entrainement des modèles.

```
La colonne 'Id' a été supprimée.

Nouvelle dimension du dataset : (1458, 239)
```

V.1. Arbres de décisions (decision trees)

V.1.1. Définition

Un arbre de décision est un modèle d'apprentissage supervisé composé d'un ensemble de "questions" organisées de manière hiérarchique en forme d'arbre. Les questions sont généralement appelées condition, fractionnement ou test. Chaque nœud non feuille contient une condition, et chaque nœud feuille contient une prédiction.

Exemple

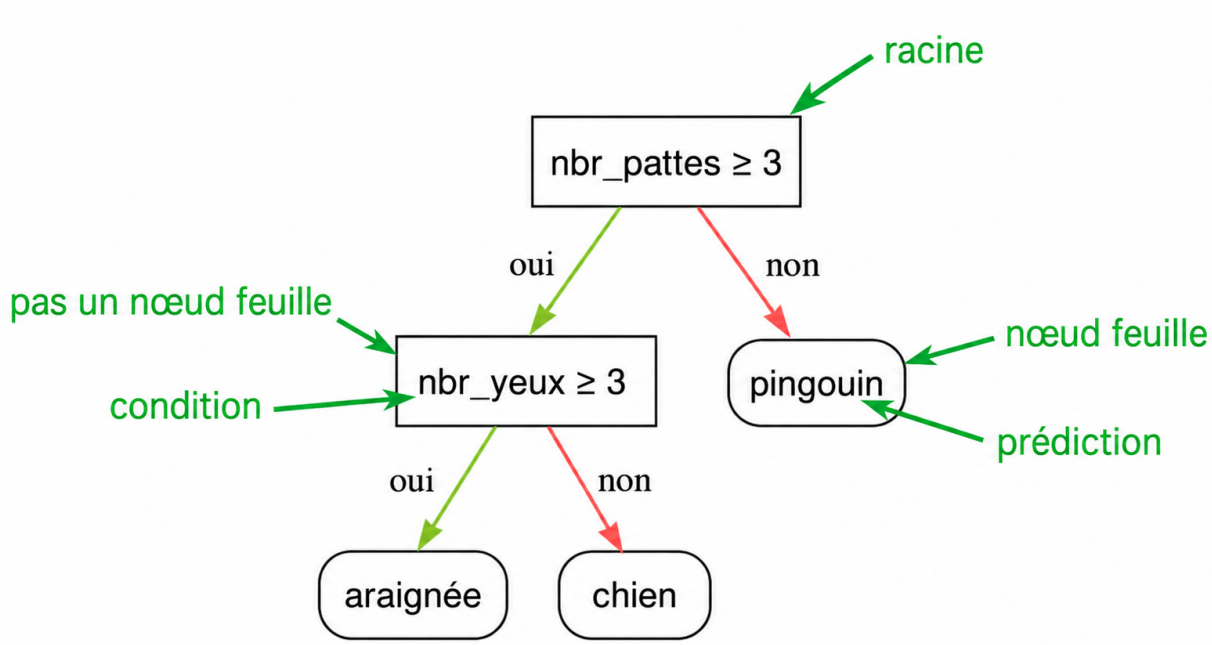


FIGURE 20 – Exemple d'arbre de décision de classification simple

Le principe est de diviser progressivement le jeu de données en sous-groupes de plus en plus homogènes en utilisant les variables explicatives. À chaque étape, l'algorithme choisit la règle de séparation qui permet d'obtenir les groupes les plus cohérents possibles.

Selon la nature de la variable que l'on souhaite prédire, on distingue deux types d'arbres.

V.1.2. Les arbres de classification

Ils sont utilisés lorsque la variable cible est qualitative.

Exemples :

- spam / non spam ;
- malade / sain ;
- client fidèle / non fidèle ;
- réussite / échec.

La feuille de l'arbre contient alors une classe.

V.1.3. Les arbres de régression

Ils sont utilisés lorsque la variable cible est quantitative, c'est-à-dire un nombre réel.

Exemples :

- prix d'une maison ;
- salaire ;
- consommation électrique ;
- chiffre d'affaires ;
- température.

La feuille de l'arbre contient alors une valeur numérique, généralement la moyenne des observations présentes dans cette feuille.

C'est ce type d'arbre qui nous intéresse ici.

V.1.4. Principe de fonctionnement d'un arbre de régression

Un arbre de régression est un modèle prédictif qui cherche à estimer une variable numérique en partitionnant progressivement les observations en groupes homogènes.

À chaque nœud, l'algorithme choisit :

- la variable explicative,
- ainsi que le seuil de séparation,

qui permettent de réduire au maximum la variabilité des valeurs de la variable cible à l'intérieur de chaque groupe.

Une fois l'arbre construit, toute nouvelle observation est dirigée de branche en branche jusqu'à une feuille, où la prédiction est simplement la valeur associée à cette feuille.

Exemple : Prédiction du prix d'une maison

Supposons que l'on dispose du dataset immobilier fictif suivant :

Maison	Surface (m^2)	Nb pièces	Quartier	Âge (ans)	Dernière rénovation (ans)	Qualité (1–10)	Prix (€)
M1	70	3	Centre	5	2	9	310 000
M2	85	4	Centre	12	3	8	340 000
M3	65	3	Nord	30	20	5	180 000
M4	120	5	Centre	8	2	10	510 000
M5	150	6	Sud	10	5	9	560 000
M6	90	4	Nord	28	15	6	240 000
M7	110	5	Centre	15	4	8	430 000
M8	55	2	Nord	35	25	4	150 000
M9	140	6	Sud	6	2	9	590 000
M10	100	5	Centre	18	8	7	370 000

TABLE 2 – Jeu de données immobilier fictif

Pour séparer les données, l'arbre va tester énormément de possibilité, comme par exemple :

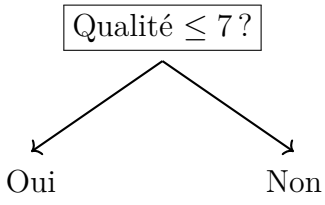
- Surface ≤ 80 ?
- Surface ≤ 100 ?
- Qualité ≤ 6 ?
- Qualité ≤ 8 ?
- Quartier = Centre ?
- Quartier = Nord ?
- Âge ≤ 15 ?
- etc.

Pour chacune de ces possibilités, il calcule laquelle produit les groupes de maisons les plus homogènes en termes de prix.

On remarque par exemple que globalement les logements de faible qualité valent beaucoup moins cher et que les logements de bonne qualité valent beaucoup plus cher.
Donc l'algorithme peut par exemple décidé de faire comme première condition pour séparer les données :

$$\boxed{\textit{Qualité} \leq 7 \text{ ?}}$$

On obtient alors deux premiers groupes de données beaucoup plus homogènes.



Première séparation

Sur la branche de gauche on a :

Maisons :

M3 : 180 000 €

M6 : 240 000 €

M8 : 150 000 €

M10 : 370 000 €

Ces prix restent encore assez dispersés. L'algorithme va donc continuer.

Sur la branche de droite, les maisons sont :

M1 : 310 000 €

M2 : 340 000 €

M4 : 510 000 €

M5 : 560 000 €

M7 : 430 000 €

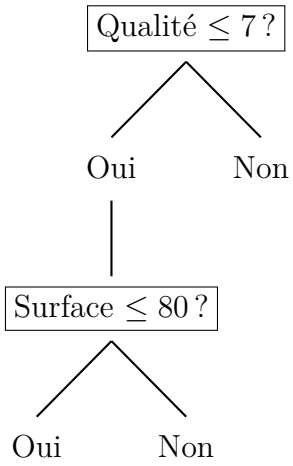
M9 : 590 000 €

Là aussi, les prix sont variés. L'arbre poursuit les séparations.

Deuxième séparation

Pour les logements de faible qualité (branche gauche), supposons que le meilleur critère soit : $\boxed{Surface \leq 80\ ?}$

L'arbre se poursuit donc ainsi :



Pour premier sous-groupe (branche gauche) on a :

M3 : 180 000 €

M8 : 150 000 €

Ces deux maisons sont très proches. Le prix moyen vaut :

$\frac{180000+150000}{2} = 165000$

La feuille prédira donc :

165 000 €

Pour le deuxième sous-groupe (branche droite) on a :

M6 : 240 000 €

M10 : 370 000 €

Les prix sont encore un peu dispersés. L'arbre continue.

Meme principe pour les logements de bonne qualité. Sur la branche droite, l'arbre pourrait choisir : $\boxed{Surface \leq 130}$

Les maisons seraient réparti ainsi :

Sur le sous-groupe de gauche (Surface ≤ 130) :

M1 : 310 000 €

M2 : 340 000 €

M4 : 510 000 €

M7 : 430 000 €

Les prix sont encore dispersés. L'arbre va continuer.

Et sur le sous-groupe de droite (Surface > 130) :

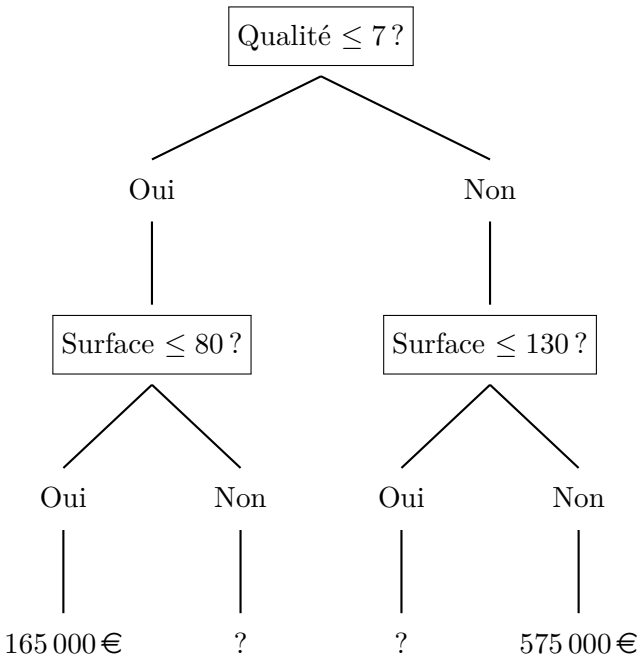
M5 : 560 000 €

M9 : 590 000 €

Ce sous-groupe est très homogène. La feuille prédira donc :

$\frac{560000+590000}{2} = 575000$

Jusqu'ici, on a l'arbre suivant :



Troisième séparation

Sur le groupe Qualité ≤ 7 puis Surface > 80 (M6, M10), l'arbre va tester plusieurs possibilité.

Supposons que la meilleure soit : $\boxed{Quartier = Centre?}$

On aura alors :

Quartier = Centre

M10 uniquement : 370 000 €

La prédiction devient :

370 000 €

Quartier $\neq$ Centre

M6 uniquement : 240 000 €

La prédiction devient :

240 000 €

Sur le semi chemin $\boxed{\text{Qualité} > 7}$ puis $\boxed{\text{Surface} \leq 130}$ (M1, M2, M4, M7), l'arbre va aussi tester plusieurs possibilité.

Supposons que la meilleure soit : $\boxed{Nb\ pièces \leq 4\ ?}$

On aura alors :

Nb pièces ≤ 4 :

M1 : 310 000 €

M2 : 340 000 €

La feuille prédira donc :

$\frac{310000+340000}{2} = 325000$ €

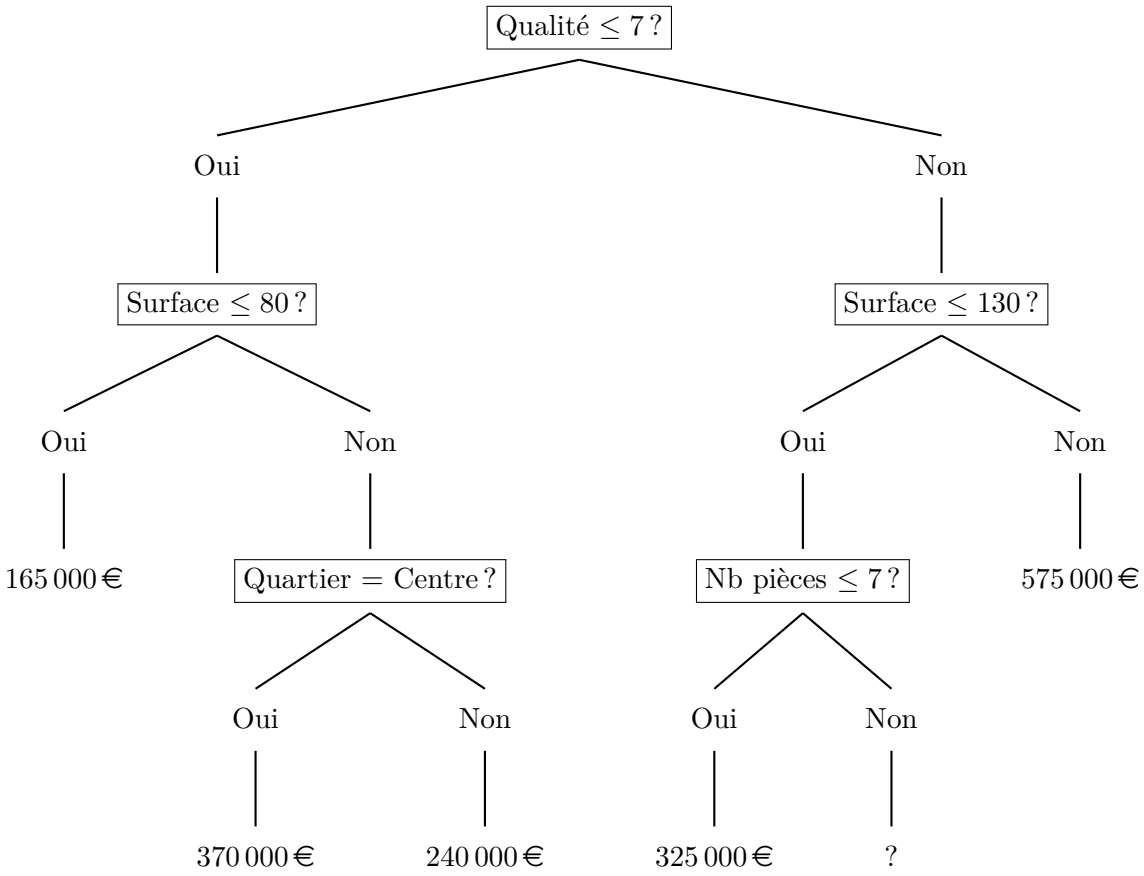
Nb pièces > 4 :

M4 : 510 000 €

M7 : 430 000 €

Les prix sont encore dispersés. L'arbre continue

Jusqu'ici, on a l'arbre suivant :



Quatrième et dernière séparation

Sur le groupe $\boxed{\text{Qualité} > 7}$ puis Surface ≤ 130 puis $\boxed{\text{Nb pièces} > 7}$ (M4, M7), l'arbre pourra par exemple choisir la condition : $\boxed{Age \leq 8}$

On aura alors :

Age ≤ 8 :

M4 uniquement : 510 000 €

La prédiction devient :

510 000 €

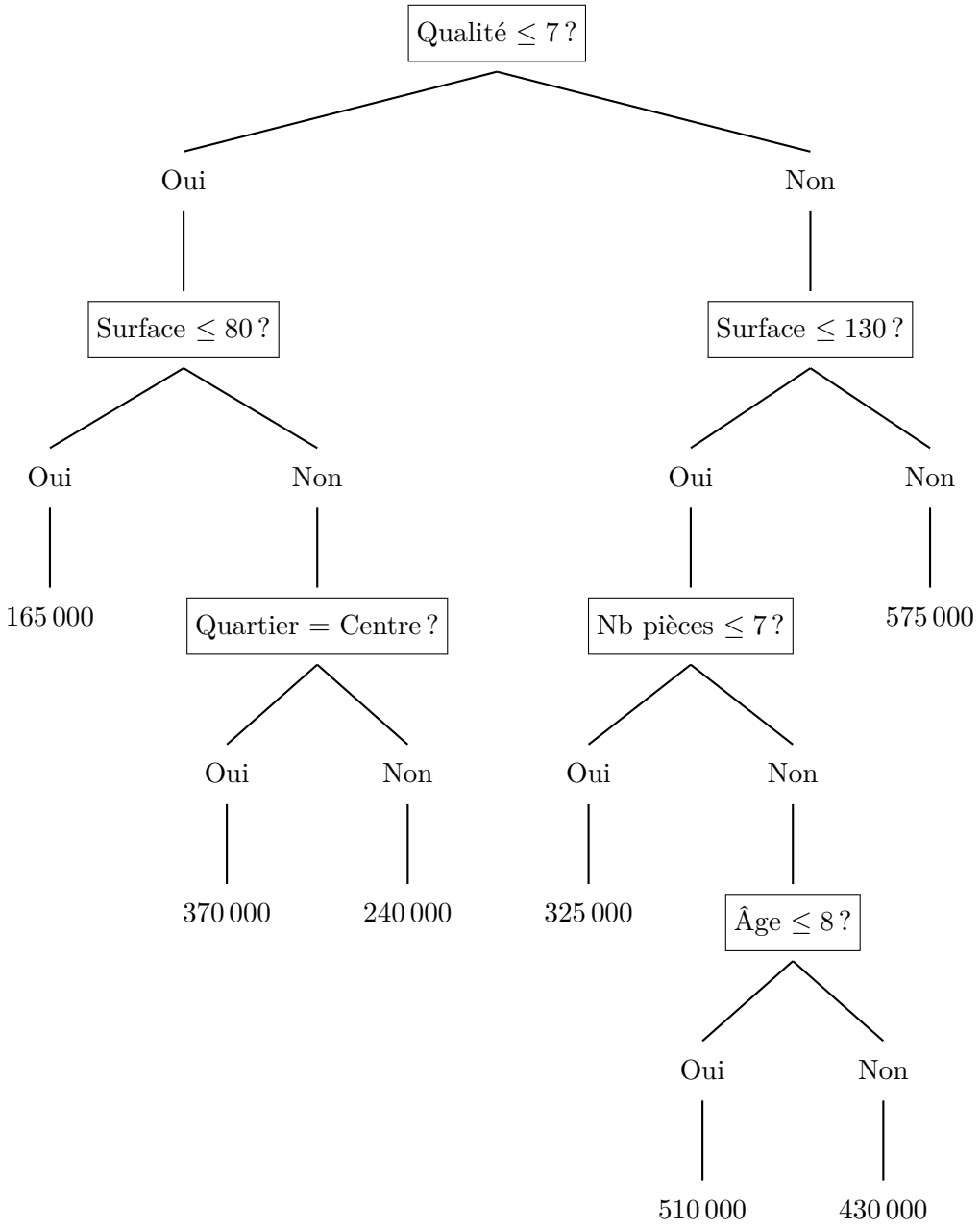
Age > 8 :

M7 uniquement : 430 000 €

La prédiction devient :

430 000 €

L'arbre est terminé et on a :



Supposons maintenant qu'on cherche à connaître le prix d'une nouvelle maison possédant les caractéristiques suivantes :

Variable	**Valeur**
Surface	115 m^2
Nombre de pièces	7
Quartier	Centre
Âge	5 ans
Dernière rénovation	3 ans
Qualité	8

TABLE 3 – Caractéristiques de la nouvelle maison à prédire

L'arbre prendra donc le chemin $\boxed{\text{Qualité} > 7\,?}$ puis $\boxed{\text{Surface} \leq 130}$ puis $\boxed{\text{Nb pièces} \leq 7}$ et prédira 325 000 pour le prix de cette maison.

V.2. Foret Aléatoire (Random Forest)

V.2.1. Définition

Une forêt aléatoire (Random Forest) est un algorithme d'apprentissage supervisé où une collection d'arbre de décision est entraîné de façon indépendante.
Plutôt que de s'appuyer sur un seul arbre, la forêt combine les prédictions de plusieurs arbres afin d'obtenir une prédiction finale généralement plus précise, plus stable et moins sensible aux variations des données.
Cet algorithme se distingue par sa grande résistance au surapprentissage (overfitting), même dans des situations extrêmes, par exemple lorsqu'il y a plus de variables explicatives que d'exemples d'entraînement.
Il s'agit probablement de l'algorihme d'entraînement des forets de décision (Decision Forests) le plus connu et le plus utilisé.

V.2.2. Structure

Une foret aléatoire est composée d'un ensemble d'arbres de décision, généralement plusieurs dizaines, plusieurs centaines, voire plusieurs milliers selon le problème étudié.

Par exemple :

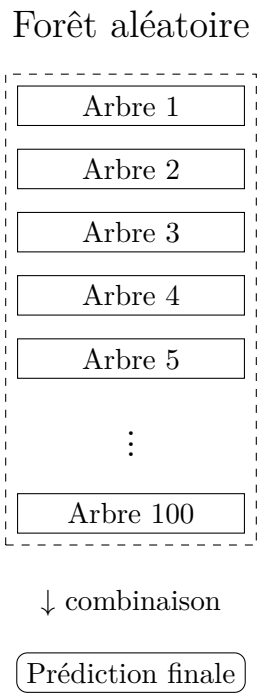


FIGURE 21 – Principe général d'une forêt aléatoire.

Chaque arbre est construit indépendamment des autres.

V.2.3. Problème que peut résoudre une foret aléatoire

Comme les arbres de décision, une forêt aléatoire peut être utilisée pour deux grandes familles de problèmes : les problèmes de classification et les problèmes de régression.

Le problème est de type classification lorsque la variable cible est qualitative.

Exemples :

- spam / non spam ;
- malade / sain ;
- fraude / non fraude ;
- réussite / échec.

Dans ce cas, lors de la prédiction, chaque arbre vote pour une classe.
La classe obtenant le plus grand nombre de votes devient la prédiction finale.

Le problème est de type régression lorsque la variable cible est numérique.

Exemples :

- prix d'une maison ;
- salaire ;
- consommation électrique ;
- chiffre d'affaires.

Dans ce cas, lors de la prédiction, chaque arbre fournit une valeur numérique.
La prédiction finale est obtenue en calculant la moyenne des prédictions de tous les arbres.
C'est ce cas qui nous intéressera dans cette étude.

V.2.4. Principe de construction des arbres d'une foret aléatoire

L'un des aspects essentiels d'une forêt aléatoire est que tous les arbres ne sont pas entraînés sur exactement les mêmes données.
Pour obtenir des arbres différents, l'algorithme introduit deux sources principales d'aléa.

La première source d'aléa est le **bootstrap** (échantillonnage avec remise).
Pour construire chaque arbre, l'algorithme réalise un tirage aléatoire avec remise. Autrement dit pour un arbre donné : certaines maisons pourront apparaître plusieurs fois alors que d'autres ne seront tout simplement pas sélectionnées du tout.
Le premier arbre est donc entraîné sur un jeu de données légèrement différent du jeu initial. Ensuite, un nouveau tirage est effectué pour construire le deuxième arbre. Puis un troisième. Puis un quatrième. Et ainsi de suite.
Chaque arbre est alors entraîné sur un ensemble d'observations différent.

La seconde source d'aléa concerne la **sélection des variables**. Elle intervient au moment où l'arbre choisit les variables servant à effectuer une séparation. Pour un arbre donné, lors de la construction de chaque nœud, l'algorithme ne considère pas toutes les variables disponibles. Il sélectionne aléatoirement un sous-ensemble de variables, puis cherche la meilleure séparation uniquement parmi celles-ci.

V.2.5. La prédiction finale

Une fois tous les arbres construits, ils sont prêts à prédire.
Lorsqu'une nouvelle observation arrive, elle est envoyée dans chaque arbre.
Chaque arbre produit alors sa propre prédiction.

Dans le cas d'une classification, chaque arbre prédit une classe et la classe obtenant le plus grand nombre de votes devient la prédiction finale.
Dans le cas d'une regression, celui qui nous interesse, chaque arbre fournit une valeur numérique et la prédiction finale est obtenue en calculant la moyenne des prédictions de tous les arbres.

Schéma résumé d'un random forest

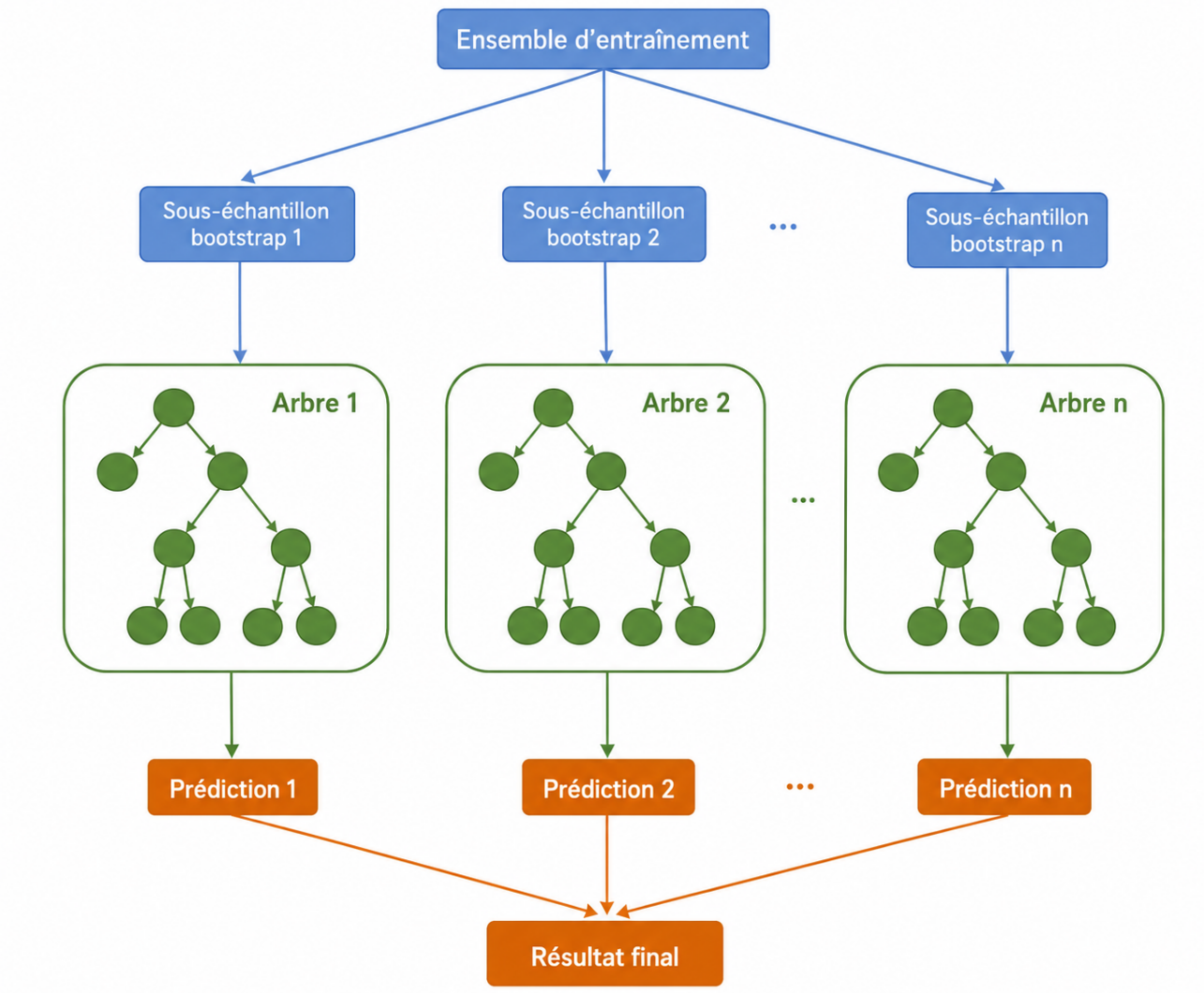


FIGURE 22 – Schéma résumant les étapes principales d'un random forest

Exemple

Reprenons le jeu de donné fictif précédent :

Maison	Surface (m^2)	Nb pièces	Quartier	Âge (ans)	Dernière rénovation (ans)	Qualité (1–10)	Prix (€)
M1	70	3	Centre	5	2	9	310 000
M2	85	4	Centre	12	3	8	340 000
M3	65	3	Nord	30	20	5	180 000
M4	120	5	Centre	8	2	10	510 000
M5	150	6	Sud	10	5	9	560 000
M6	90	4	Nord	28	15	6	240 000
M7	110	5	Centre	15	4	8	430 000
M8	55	2	Nord	35	25	4	150 000
M9	140	6	Sud	6	2	9	590 000
M10	100	5	Centre	18	8	7	370 000

TABLE 4 – Jeu de données immobilier fictif

Supposons que nous décidions de construire une forêt aléatoire composée de 3 arbres (très peu nombreux, c'est juste pour illustrer. En pratique, on en utilise souvent beaucoup plus).

Étape 1 : création des échantillons bootstrap

Pour chacun des 3 arbres, l'algorithme crée un nouvel échantillon par tirage aléatoire avec remise.

Par exemple :

Arbre 1 : M2, M5, M5, M1, M8, M10, M4, M7, M7, M3

Arbre 2 : M4, M1, M6, M9, M2, M2, M8, M10, M5, M3

Arbre 3 : M1, M3, M3, M4, M7, M8, M9, M10, M6, M2

Chaque arbre reçoit donc un jeu de données légèrement différent.

Étape 2 : construction de chaque arbre

Chaque arbre est ensuite construit indépendamment.
À chaque nœud, il ne cherche pas la meilleure séparation parmi toutes les variables disponibles, mais uniquement parmi un sous-ensemble de variables choisi aléatoirement.

Arbre 1

Premier nœud :

Variables disponibles : toutes les variables du jeu de données

Variables regardées :

- Surface
- Qualité
- Âge

Il choisit par exemple :

Qualité ≤ 7 ?

Arbre 2

Premier nœud :

Variables disponibles : toutes les variables du jeu de données

Variables regardées :

- Quartier
- Surface
- Nombre de pièces

Il choisit par exemple :

Surface ≤ 95 ?

Arbre 3

Premier nœud :

Variables disponibles : toutes les variables du jeu de données

Variables regardées :

- Qualité
- Ancienneté de la rénovation
- Quartier

Il choisit par exemple :

Quartier = Centre ?

Ainsi, les arbres n'ont pas la même structure et ne prennent pas exactement les mêmes décisions, même s'ils apprennent à partir du même jeu de données d'origine.

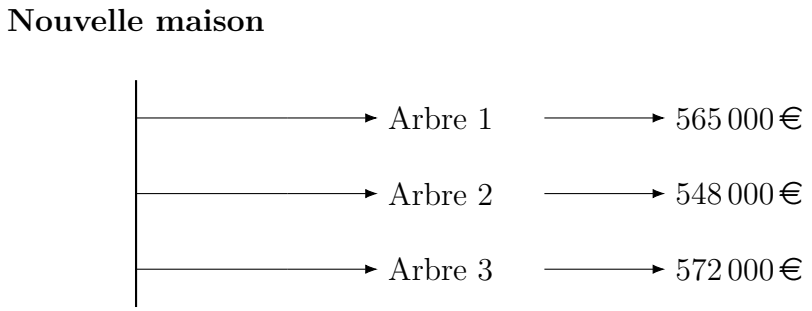
Étape 3 : prédiction d'une nouvelle maison

Maintenant supposons qu'on cherche à connaitre le prix d'une nouvelle maison présentant les caractéristiques suivantes :

Variable	**Valeur**
Surface	145 m^2
Nombre de pièces	6
Quartier	Sud
Âge	8 ans
Dernière rénovation	2 ans
Qualité	9

TABLE 5 – Caractéristiques de la nouvelle maison à prédire.

Cette nouvelle maison est envoyée dans chacun des arbres de la forêt et chaque arbre suit son propre chemin, selon les règles qu'il a apprises.
A la fin, chaque arbre fournit donc sa propre estimation du prix.
On peut se retrouver par exemple avec :



a forêt aléatoire calcule ensuite la moyenne des prédictions obtenues :

$$\frac{565000 + 548000 + 572000}{3} \approx 561\ 667$$

La prédiction finale est donc :

$$\boxed{561\ 667 \text{ euros}}$$

V.3. Entrainement des modèles

Avant toute chose, on sépare l'ensemble de données en deux parties : une partie sera utilisée pour l'entrainement des modèles et une autre pour l'évaluation finale des modèles.
Nous disposons de 1458 observations et beaucoup de variables (239). On structure la séparation en 80% des données pour l'entrainement et 20% pour l'évaluation finale.

```
Ensemble de données d'entrainement : (1166, 239)

Ensemble de données de validation : (292, 239)
```

V.3.1. Validation croisée K-Fold

Nous entrainons chaque modèle en utilisant la valisation croisée K-Fold.
La validation croisée K-Fold est une méthode utilisée pour entrainer et évaluer la performance d'un modèle de manière plus fiable qu'avec un seul découpage des données.
Dans le présent travail, l'ensemble de données est découpé en 12 groupes (ou folds) de tailles égales. Chacun des modèles est alors entrainé et évalué 12 fois.

Par exemple, sur un modèle donné :

- Au premier tour : 11 groupes sont utilisés pour l'entrainement et le groupe restant est utilisé comme ensemble de validation
- Au deuxième tour : Un autre groupe devient l'ensemble de validation et les 11 groupes restant servent pour l'entrainement
- Le processus continue jusqu'à ce que chacun des 12 groupes ait servi exactement une et une seule fois d'ensemble de validation.

On obtient un fonctionnement du style :

Tour	**Groupes d'entraînement**	**Groupe de validation**
1	Groupes 2 à 12	Groupe 1
2	Groupes 1 et 3 à 12	Groupe 2
3	Tous les groupes sauf le groupe 3	Groupe 3
⋮	⋮	⋮
12	Groupes 1 à 11	Groupe 12

TABLE 6 – Fonctionnement de la validation croisée à 12 groupes

A chaque tour, le modèle repart de zéro, c'est à dire qu'il est de nouveau entrainé sur les 11 groupes selectionnés, puis évalué sur le groupe qui n'a pas participé à cet entrainement. De ce fait, avec 12 groupes on obtient exactement :

- 12 entrainements
- 12 évaluations
- 12 scores de performance (principalement la rmsle)

qui est donnée par :

$$\text{RMSLE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}[log(y_i) - log(\hat{y}_i)]^2}$$

avec

- y_i la valeur réelle du prix
- $\hat{y}_i$ la valeur prédite du prix
- N le nombre de maisons

On calcule ensuite la moyenne des 12 scores :

$$Score\ moyen = \overline{Score} = \frac{score_1 + score_2 + score_3 + \cdots + score_{12}}{12} = \frac{1}{N}\sum_{i=1}^{N}\text{Score}_i$$

avec $N = 12$

Cette moyenne donne une estimation globale de la performance du modèle.

On peut également calculer l'écart-type des 12 scores.

$$\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(Score_i - \overline{Score})^2}$$

avec $N = 12$

Un faible écart-type indique que les scores obtenus lors des différentes évaluations sont relativement proches du score moyen. Un écart-type élevé indique qu'ils sont plutot éloignés du score moyen.

On peut remarquer que la validation croisée K-Fold est une méthode particulièrement interessante. En effet avec un seul découpage des données (du style 75% entrainement et 25% validation), le modèle n'est évalué qu'une seule fois. Le score obtenue peut alors fortement dépendre du harsard du découpage.
Alors qu'avec une validation croisé K-Fold, le modèle est évalué sur plusieurs parties différentes des données. Chaque observation ici sert une fois à la validation et 11 fois à l'entrainement.
La moyenne des 12 scores est donc généralement plus stable et plus représentative de la capacité du modèle à généraliser sur de nouvelles données.

V.3.2. Entrainement des modèles de bases (avec hyperparamètres choisis manuellement)

Nous entrainons les modèles suivants :

- Random Forest Regressor,
- Extra Trees Regressor
- Decision Tree Regressor (CART)
- Gradient Boosting Regressor
- Extreme Gradient Boosting (XGBoost)
- Ridge Regression

avec leurs principaux hyperparamètres choisis manuellement.

Les hyperparamètres sont des réglages choisis avant l'entrainement.
Les principaux hyperparamètres réglés pour chacun des modèles cités précédemment sont :

Random Forest :

- `n_estimators` : qui représente le nombre total d'arbres de la foret. Plus il y a d'arbre, plus la prédiction est généralement stable et précise, mais le temps de calcul augmente. Nous en choisissons ici 300.
- `max_features` : qui représente le nombre (ou la proportion) de variables explicatives pouvant être testées à chaque division d'un arbre. Cela introduit de la diversité entre les arbres. Ici nous utilisons 'sqrt', ce qui signifie qu'à chaque noeud l'arbre n'examine pas toutes les variables mais seulement la racine carrée du nombre total de variables.
- `random_state` : qui fixe (ou non) la graine aléatoire. Cela permet de reproduire exactement les mêmes résultats d'une exécution à l'autre. Nous choisissons ici la valeur 42.
- `n_jobs` : qui est le nombre de coeurs du processeur utilisés pour entraîner les arbres. La valeur 'None' indique que un seul coeur est utilisé. La valeur '-1' indique que tous les coeurs disponibles sont utilisés, ce qui accélère l'entraînement. Nous choisissons la valeur '-1'.

Et les autres hyperparamètres ont été laissé avec leur valeur par défaut.

Decision Tree Regressor (CART) :

- `random_state` $= 42$

Et les autres hyperparamètres ont été laissé avec leur valeur par défaut. Par exemple :

- `max_depth` : qui représente la profondeur maximale de l'arbre à 'None' comme valeur par défaut. Cela signifie que l'arbre continue à se développer jusqu'à ce qu'il ne puisse plus être diviser.
- `min_samples_split` : qui représente le nombre minimal d'observations nécessaires pour diviser un noeud. Sa valeur par défaut est 2, ce qui empêche de créer des divisions lorsque le noeud contient moins de 2 observations.
- `min_samples_leaf` : qui représente le nombre minimal d'observations que doit contenir une feuille. Sa valeur par défaut est 1.

Extra Trees Regressor :

Mêmes valeurs que pour Random Forest :

- `n_estimators` $= 300$
- `max_features` = `'sqrt'`
- `random_state`$= 42$
- `n_jobs` $= -1$

Gradient Boosting Regressor :

- `n_estimators` : qui est le nombre total d'arbres construits successivement. Chaque nouvel arbre apprend à corriger les erreurs des arbres précédents. Plus `n_estimators` est grand, plus le modèle peut apprendre, mais le temps d'entraînement augmente et le risque de surapprentissage peut aussi augmenter. Nous choisissons `n_estimators` $= 500$.
- `max_depth` : qui est la profondeur maximale de chaque arbre. Nous laissons ici la valeur par défaut qui est 3.
- `learning_rate` : qui est le taux d'apprentissage. Il controle l'importance de la correction apportée par chaque nouvel arbre. Une valeur faible rend souvent l'apprentissage plus robuste mais necessite davantage d'arbre. Nous prenons ici la valeur $0,05$.
- `loss` : qui est la fonction de perte utilisée pendant l'entraînement. Nous laissons la perte par défaut qui est la '`squared_error`', c'est à dire l'erreur quadratique moyenne (MSE)
- `random_state` $= 42$
- `n_jobs` $= -1$

Extreme Gradient Boosting (XGBoost) :

- `n_estimators` $= 300$
- `random_state` $= 42$
- `tree_method` : qui représente l'algorithme utilisé pour rechercher les meilleurs coupures dans les arbres. Nous choisisssons la valeur 'hist', ce qui signifie que les valeurs sont regroupées en intervalles avant de rechercher les coupures.
- `verbosity` : qui controle la quantité de messages affichés pendant l'entraînement. Nous choisissons la valeur '0' qui indique qu'aucun message ne sera affiché.
- `n_jobs` $= -1$

Ridge Regression :

- `alpha` : qui est le coefficient de régulation. Il controle l'intensité de la pénalisation appliquée aux coefficients du modèle. Nous choisissons `alpha` = 10

Et les autres hyperparamètres ont été laissé avec leur valeur par défaut.

Chacun des modèles précédents est entraîné en utilisant le principe de validation croisée décrit plus haut et avec les paramètres fixés précédemment. On compare les métriques des différents modèles obtenus ci-dessous :

Modèle	RMSLE moyenne	Écart-type
Random Forest Regressor	0.14217	0.02085
Extra Trees Regressor	0.14900	0.02088
Decision Tree Regressor (CART)	0.20143	0.02532
Gradient Boosting Regressor	0.12018	0.01425
XGBoost Regressor	0.13440	0.01873
Ridge Regression	0.11173	0.01423

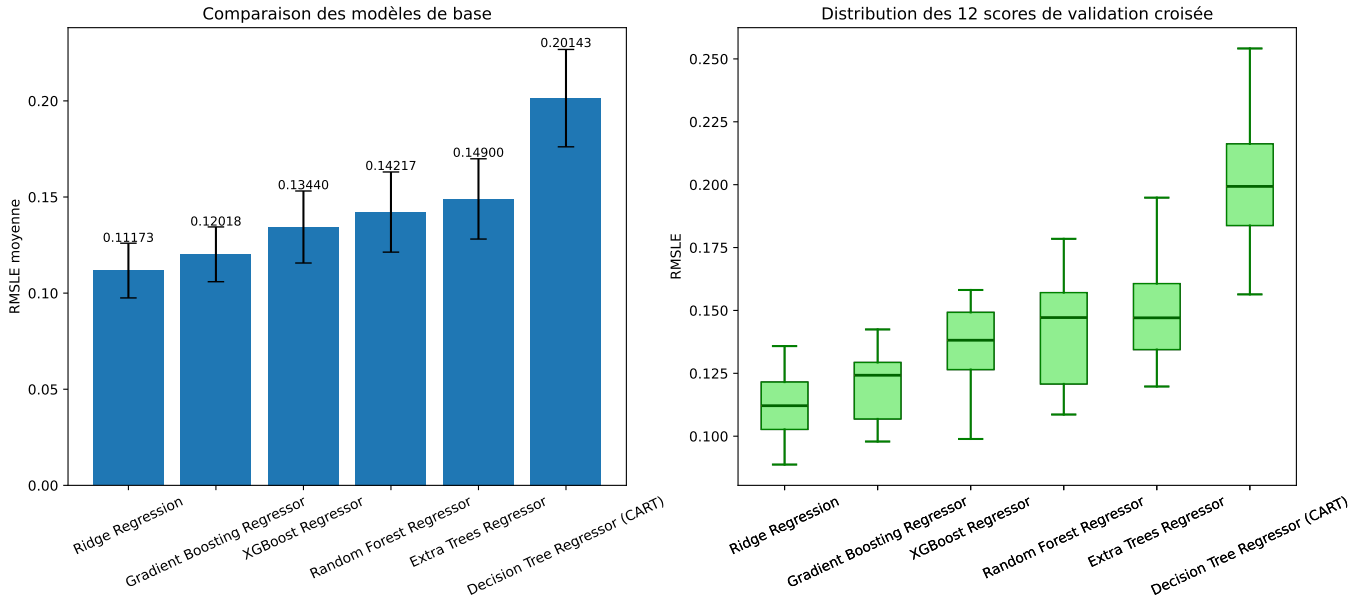


FIGURE 23 – Classement des modèles

```
Meilleur modèle de base : Ridge Regression

RMSLE moyenne : 0.11173
```

On observe que le Decision Tree Regressor (CART) présente la RMSLE moyenne la plus élevée parmi tous les modèles évalués. Son boxplot met également en évidence une plus grande dispersion des scores obtenus lors de la validation croisée, traduisant une variabilité plus importante de ses performances en fonction des différents groupes de validation.
Ces résultats sont cohérents avec le fonctionnement d'un arbre de décision unique, puisque ces

prédictions reposent sur une seule structure, sans possibilité de compenser ses erreurs par d'autres arbres, ce qui rend le modèle généralement moins précis et moins stable que les méthodes reposant sur plusieurs arbres telles que les forêts aléatoires ou le gradient boosting.

Dans la suite de cette étude, nous montrerons qu'il est possible d'améliorer les performances de prédiction de chacun des modèles précédents en appliquant des étapes supplémentaires de transformations sur le dataset.

Les sections qui vont suivre sont consacrées à la création de nouvelles variables et à la correction de l'asymétrie de certaines variables numériques, puis nous évaluerons leur impact sur les performances des différents modèles précédents.

VI. Création de nouvelles variables (Feature Creation)

VI.1. Feature Creation

Les variables brutes ne représentent pas toujours ce que regarde directement un acheteur. Dans cette partie, nous créons de nouvelles variables qui transforment les données brutes en informations plus pertinentes et explicative pour la prédiction du prix du logement. Autrement dit, on va résumer dans une seule variable une information qui est dispersée dans plusieurs autres variables.

Par exemple, le dataset contient les variables :

- 1stFlrSF : surface du rez-de-chaussée
- 2ndFlrSF : surface du premier étage
- TotalBsmtSF : surface total du sous-sol

On créer alors la variable surface total de la maison (TotalSF) qui est telle que :

$$\text{TotalSF} = \text{TotalBsmtSF} + \text{1stFlrSF} + \text{2ndFlrSF}$$

Cette variable est plus informative sur le prix que chacune des trois prises séparément.

De même, le dataset contient les variables :

- FullBath : nombre de salle de bain complètes (hors sous-sol)
- HalfBath : nombre de demi-salle de bain (hors sous-sol)
- BsmtFullBath : nombre de salle de bain complètes situées dans le sous-sol
- BsmtHalfBath : nombre de demi-salles de bain situées dans le sous-sol

qui peuvent être résumer dans la variable TotalBathrooms qui représente le nombre total de salle de bains de la maison et qui est telle que :

$$\text{TotalBathrooms} = \text{FullBath} + 0.5 \times \text{HalfBath} + \text{BsmtFullBath} + 0.5 \times \text{BsmtHalfBath}$$

avec un poids de $0,5$ pour les demi-salle de bain.
Le nombre total de salle de bain de la maison est beaucoup plus parlant.

Pareillement, on sait qu'en règle générale, l'age de la maison est souvent plus pertinente pour prédire le prix que l'année de construction ou l'année de vente, c'est pourquoi nous créons une nouvelle variable Age de la maison (HouseAge) qui est telle que :

$$\text{HouseAge} = \text{YrSold} - \text{YearBuilt}$$

L'information relative aux porches peut également être résumée. En effet, le dataset contient les variables :

- OpenPorchSF : surface des porches ouverts
- EnclosedPorch : surface des porches fermées
- 3SsnPorch : surface d'un porche trois saisons
- ScreenPorch : surface d'un porche moustiquaire (ou grillage)
- WoodDeckSF : surface de la terrasse en bois

La nouvelle variable TotalporchSF qui représente la surface total des espaces extérieurs aménagés et utilisables de la maison est donnée par :

$$\text{TotalPorchSF} = \text{OpenPorchSF} + \text{EnclosedPorch} + \text{3SsnPorch} + \text{ScreenPorch} + \text{WoodDeckSF}$$

Cette surface total est plus directement liée au prix que chacune des surfaces prises séparément.

De même, on a les variables :

- YearRemodAdd : année de la dernière rénovation
- YrSold : année de vente

La nouvelle variable RemodAge qui représente l'ancienneté de la dernière rénovation au moment de la vente est donnée par :

$$\text{RemodAge} = \text{YrSold} - \text{YearRemodAdd}$$

Ensuite on peut créer des variables binaires qui renseignent pour la caractéristique concernée, si la maison la possède ou pas.

Par exemple :

- la variable HasPool, qui vaut 1 si la maison possède une piscine (PoolArea > 0) et 0 sinon.
- la variable IsRemodeled qui vaut 1 si la maison a été rénovée (YearRemodAdd $\neq$ YearBuilt) et 0 sinon.
- la variable IsNew qui vaut 1 si la maison est neuve (YrSold=YearBuilt) et 0 sinon.
- la variable HasGarage qui vaut 1 si la maison possède un 1 garage (GarageArea > 0) et 0 sinon.
- la variable HasBsmt qui vaut 1 si la maison possède un sous-sol (TotalBsmtSF > 0) et 0 sinon.

- la variable HasFireplace qui vaut 1 si la maison possède au moins une alarme incendie (Fireplaces > 0) et 0 sinon.
- Et enfin la variable Has2ndFlr qui vaut 1 si la maison possède un 1er étage (2ndFlr > 0) et 0 sinon.

```
Nombre de nouvelles variables créées : 12

Liste des nouvelles variables :
1. TotalSF
2. TotalBathrooms
3. TotalPorchSF
4. HouseAge
5. RemodAge
6. IsRemodeled
7. IsNew
8. HasPool
9. HasGarage
10. HasBsmt
11. HasFireplace
12. Has2ndFlr

Nouvelle dimension du dataset : (1458, 252)
```

Nous affichons ci-dessous un bref aperçu des nouvelles variables crées ci-dessus

TotalSF	TotalBathrooms	TotalPorchSF	HouseAge	RemodAge	IsRemodeled	IsNew	HasPool	HasGarage	HasBsmt	HasFireplace	Has2ndFlr
2566	3.5	61	5	5	0	0	0	1	1	0	1
2524	2.5	298	31	31	0	0	0	1	1	1	0
2706	3.5	42	7	6	1	0	0	1	1	1	1
2473	2.0	307	91	36	1	0	0	1	1	1	1
3343	3.5	276	8	8	0	0	0	1	1	1	1
2158	2.5	390	16	14	1	0	0	1	1	0	1
3380	3.0	312	3	2	1	0	0	1	1	1	0
3197	3.5	667	36	36	0	0	0	1	1	1	1
2726	2.0	295	77	58	1	0	0	1	1	1	1
2068	2.0	4	69	58	1	0	0	1	1	1	0

FIGURE 24 – Aperçu des nouvelles variables crées

VI.2. Analyse des corrélations

En incluant les nouvelles variables crées précédemment, nous recherchons à nouveau à connaître parmi toutes les variables explicatives du dataset, lesquelles sont celles qui sont les plus fortement corrélées avec la variable cible SalePrice.

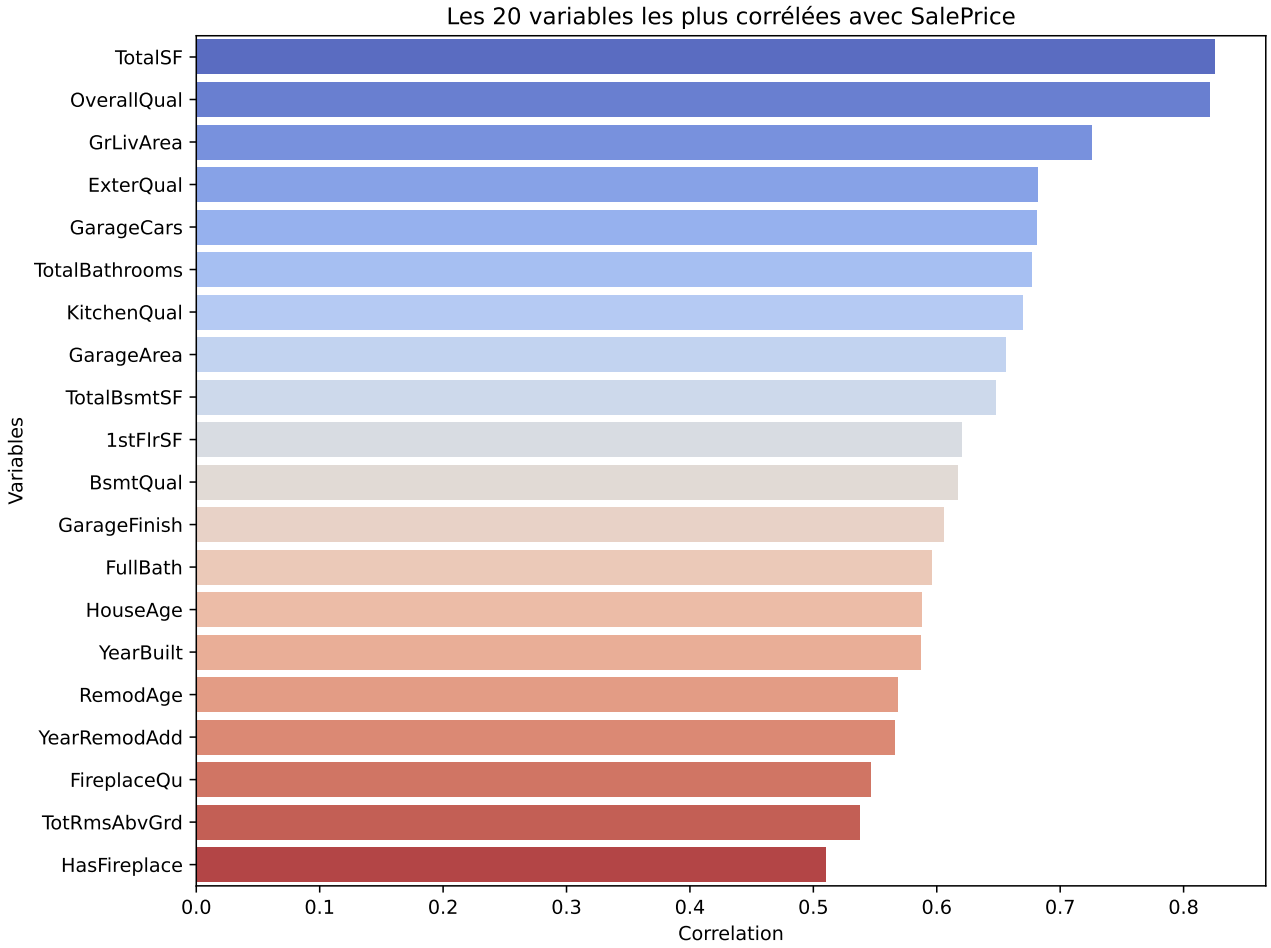


FIGURE 25 – Top 20 des variables les plus corrélées avec SalePrice

On peut constater que certaines des nouvelles variables que l'on vient de créer sont fortement corrélées avec la variable cible. Par exemple :

- TotalSF est désormais la variable la plus corrélée avec SalePricre, devant OverallQual. Cela montre que la surface total de la maison est plus représentatif de son prix que les différentes surfaces prises séparément.
- TotalBathrooms aparaît également parmi les variables les plus corrélées avec SalePrice, ce qui montre que le nombre total de salle de bain est beaucoup plus liées au prix que les variables dénombrant séparément chaque type de salle de bain.
- HouseAge et RemoveAge figurent aussi dans le top 20 des variables les plus explicatives du prix des maisons.

On peut voir que malgré la création de nouvelles variables, OverallQuall reste l'une des variables les plus fortement liées au prix, ce qui confirme que la qualité général de la maison demeure un déterminant majeur de son prix.

Les résultats précédents montrent que les nouvelles variables crées apportent une information pertinente et justifient leur utilisation pour l'entraînement des modèles de machine learning.

VII. Correction de l'asymétrie des variables numériques quantitatives continues

VII.1. Variables asymétriques

Nous considérons les variables numériques quantitatives continues suivantes (dates exclues) :

- LotFrontge,
- LotArea,
- MasVnrArea,
- BsmtFinSF1,
- BsmtFinSF2,
- BsmtUnfSF,
- TotalBsmtSF,
- stFlrSF,
- 2ndFlrSF,
- LowQQualFinSF,
- GrLivArea,
- GarageArea,
- WoodDeckSF,
- OpenPorchSF,
- EnclosedPorch,
- 3SsnPorch,
- ScreenPorch,
- PoolArea,
- MiscVal
- TotalSF
- TotalPorchSF

Nous affichons ci-dessous les allures de leurs distributions respectives ainsi que les coefficients d'asymétries qui leur sont associés.

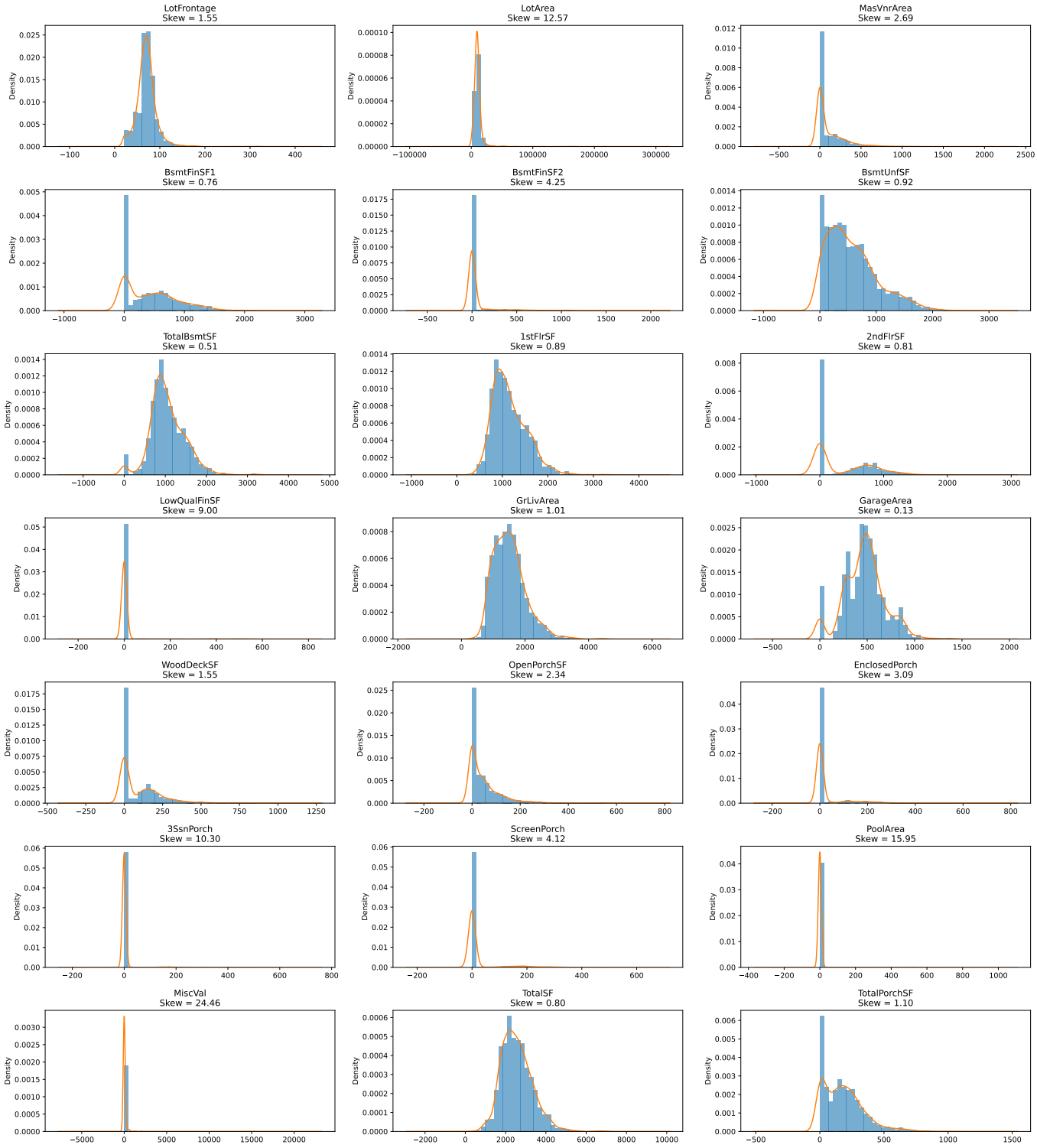


FIGURE 26 – Asymétrie des variables numériques quantitatives continues

On peut constater que la majorité de ces variables est lourdement asymétrique vers la droite. Nous appliquons une transformée de Box-Cox. Cette dernière aide énormément les modèles linéaires et peut aussi améliorer les performances des modèles basés sur les arbres.

VII.2. Correction d'asymétrie : transformation de Box-Cox

Dans cette sous-partie, la correction de l'asymétrie est effectuée uniquement sur les variables qui présentent un coefficient d'asymétrie strictement supérieur à 0.70 (skewness > 0.70).

Pour corriger l'asymétrie des variables numériques quantitatives, nous utilisons une méthode bien connue qui est la transformation de Box-Cox.

Pour toute valeur x strictement positive, on définit la transformée de Box-Cox de la manière suivante :

$$B(x,\lambda) = \begin{cases} \dfrac{x^\lambda - 1}{\lambda}, & \text{si } \lambda \neq 0, \\ \log(x), & \text{si } \lambda = 0. \end{cases}$$

- Si $\lambda > 1$ cela amplifie les grandes valeurs de x
- Si $\lambda < 1$ cela réduit les grandes valeurs de x

Cette transformation s'applique uniquement si :

- Les observations sont strictement positives
- Indépendantes et identiquement distribuées (iid)

La seconde condition est vérifiée. Cependant, certaines variables contiennent des valeurs nulles. Pour résoudre ce problème, on applique **Box-Cox** à $x+1$ qui est donnée par :

$$B(x+1,\lambda) = \begin{cases} \dfrac{(x+1)^\lambda - 1}{\lambda}, & \text{si } \lambda \neq 0, \\ \log(x+1), & \text{si } \lambda = 0. \end{cases}$$

Si l'on pose la variable transformée $y = B(x+1,\lambda)$. Pour revenir aux données normales, on cherche à trouver x en fonction de y. On distingue deux cas :

1er cas : $\lambda \neq 0$

On part de :

$$y = \frac{(x+1)^\lambda - 1}{\lambda}$$

On multiplie les deux membres par λ :

$$\lambda y = (x+1)^\lambda - 1$$

On ajoute 1 des deux côtés :

$$\lambda y + 1 = (x+1)^\lambda$$

On élève les deux membres à la puissance $\frac{1}{\lambda}$:

$$(\lambda y + 1)^{\frac{1}{\lambda}} = x + 1$$

Enfin, on soustrait 1 :

$$x = (\lambda y + 1)^{\frac{1}{\lambda}} - 1 = \exp\left(\frac{1}{\lambda}\ln(\lambda y + 1)\right) - 1.$$

avec $\lambda y + 1 > 0$

2ème cas : $\lambda = 0$:

$$y = \ln(x + 1) \Leftrightarrow e^y = x + 1 \Leftrightarrow x = e^y - 1.$$

La formule inverse complète de la transformation de Box-Cox est donc :

$$B^{-1}(y, \lambda) = \begin{cases} (\lambda y + 1)^{\frac{1}{\lambda}} - 1, & \text{si } \lambda \neq 0, \\ e^y - 1, & \text{si } \lambda = 0. \end{cases}$$

La valeur optimal de λ est estimée par la méthode du maximum de vraissemblance.

Variable	λ **optimal**
PoolArea	-38.114212
3SsnPorch	-11.655547
LowQualFinSF	-10.004913
MiscVal	-4.277596
ScreenPorch	-2.431522
BsmtFinSF2	-1.519430
EnclosedPorch	-1.417909
MasVnrArea	-0.243187
2ndFlrSF	-0.128859
WoodDeckSF	-0.080132
1stFlrSF	-0.007729
OpenPorchSF	0.011447
LotArea	0.040531
GrLivArea	0.069508
BsmtFinSF1	0.230631
TotalSF	0.356348
TotalPorchSF	0.421357
BsmtUnfSF	0.470857
LotFrontage	0.550018

TABLE 7 – Valeurs optimales du paramètre λ

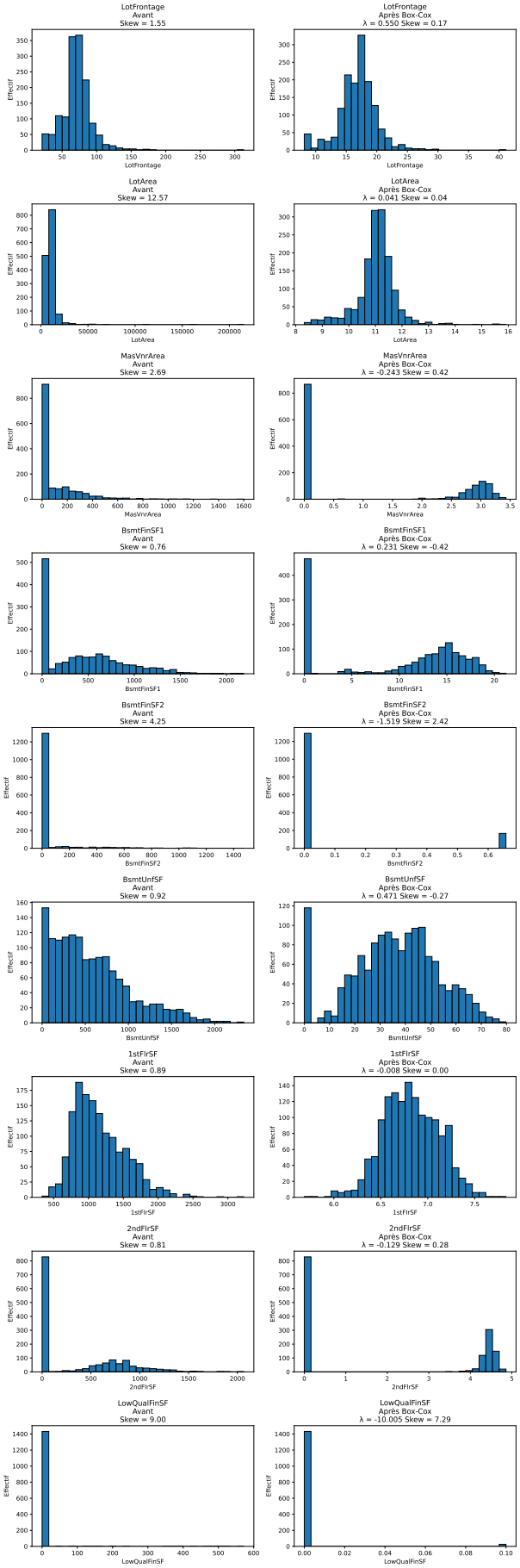


FIGURE 27 – Distributions avant et après correction (premier groupe de variable)

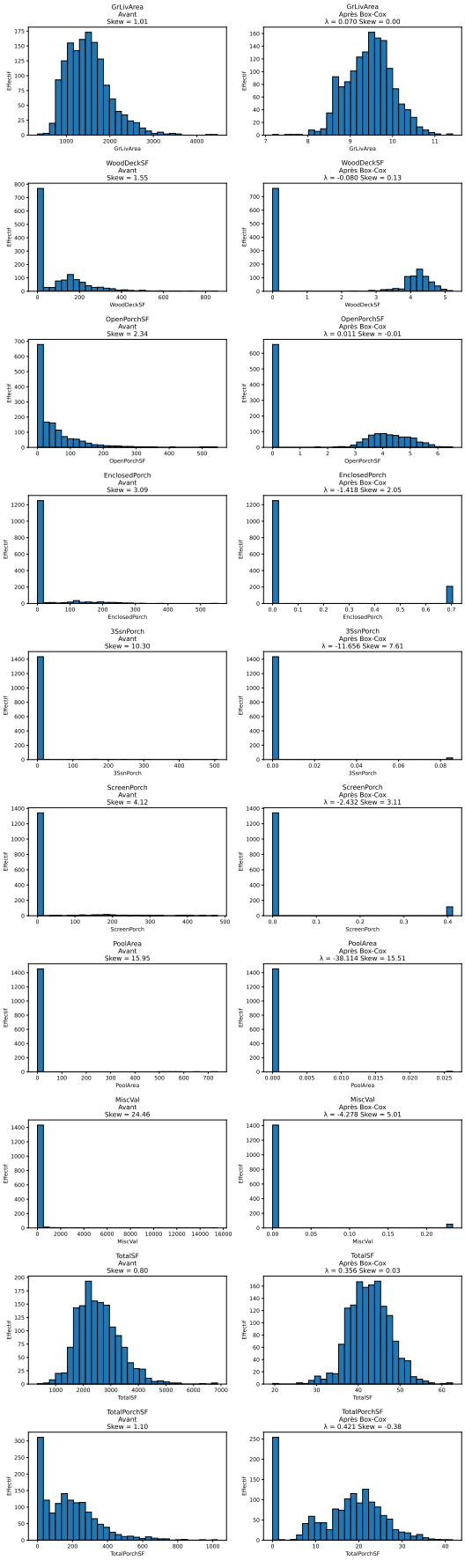


FIGURE 28 – Distributions avant et après correction (second groupe de variable)

VIII. Entrainement des modèles (Random Forest Regressor, Extra Trees Regressor, Decision Tree Regressor (CART), Gradient Boosting Regressor, Extreme Gradient Boosting (XGBoost), Ridge Regression) et évaluation après creation de nouvelles variables et correction d'asymétrie.

A nouveau, on sépare l'ensemble de données en deux parties : une partie pour l'entrainement des modèles et une autre pour l'évaluation finale des modèles.
On structure également la séparation en 80% des données pour l'entrainement et 20% pour l'évaluation finale.

```
Ensemble de données d'entrainement : (1166, 251)

Ensemble de données de validation : (292, 251)
```

VIII.1. Résultats des modèles entrainés et évalués (par cv) après correction d'asymétrie et création de nouvelles variables

Modèle	RMSLE moyenne	Écart-type
Random Forest Regressor	0.13650	0.01984
Extra Trees Regressor	0.14398	0.02136
Decision Tree Regressor (CART)	0.19569	0.02918
Gradient Boosting Regressor	0.11892	0.01651
XGBoost Regressor	0.13281	0.01736
Ridge Regression	0.11041	0.01376

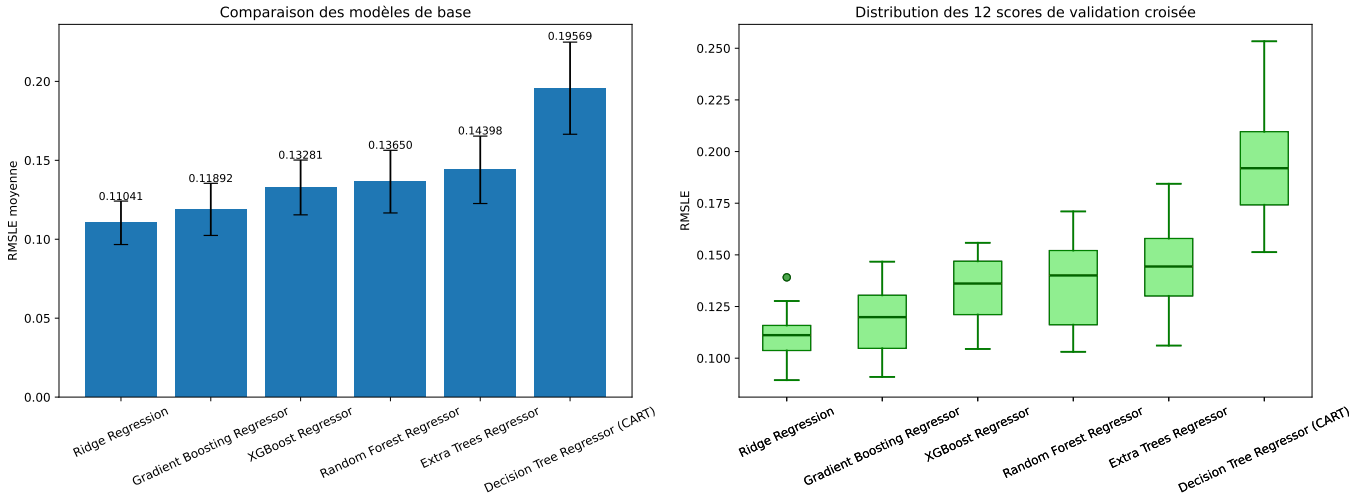


FIGURE 29 – Classement des modèles après correction de skewness et feature creation

```
Meilleur modèle de base : Ridge Regression

RMSLE moyenne : 0.11041
```

Comme on peut le voir, le meilleur modèle aprés feature creation et correction d'asymétrie reste la régression Ridge, dont l'erreur de prédiction a été améliorée.

VIII.2. Comparaison avec les modèles entainées et évalués (par cv) avant correction d'asymétrie et création de nouvelles variables

	RMSLE avant	Écart-type avant	RMSLE après	Écart-type après	Gain RMSLE	Amélioration (%)
Ridge Regression	0.11173	0.01423	0.11041	0.01376	0.00132	1.18370
Gradient Boosting Regressor	0.12018	0.01425	0.11892	0.01651	0.00127	1.05357
XGBoost Regressor	0.13440	0.01873	0.13281	0.01736	0.00159	1.18054
Random Forest Regressor	0.14217	0.02085	0.13650	0.01984	0.00568	3.99221
Extra Trees Regressor	0.14900	0.02088	0.14398	0.02136	0.00502	3.37166
Decision Tree Regressor (CART)	0.20143	0.02532	0.19569	0.02918	0.00574	2.85046

FIGURE 30 – Tableau de comparaison des performanes des modèles avant et après correction d'asymétrie et création de nouvelles variables

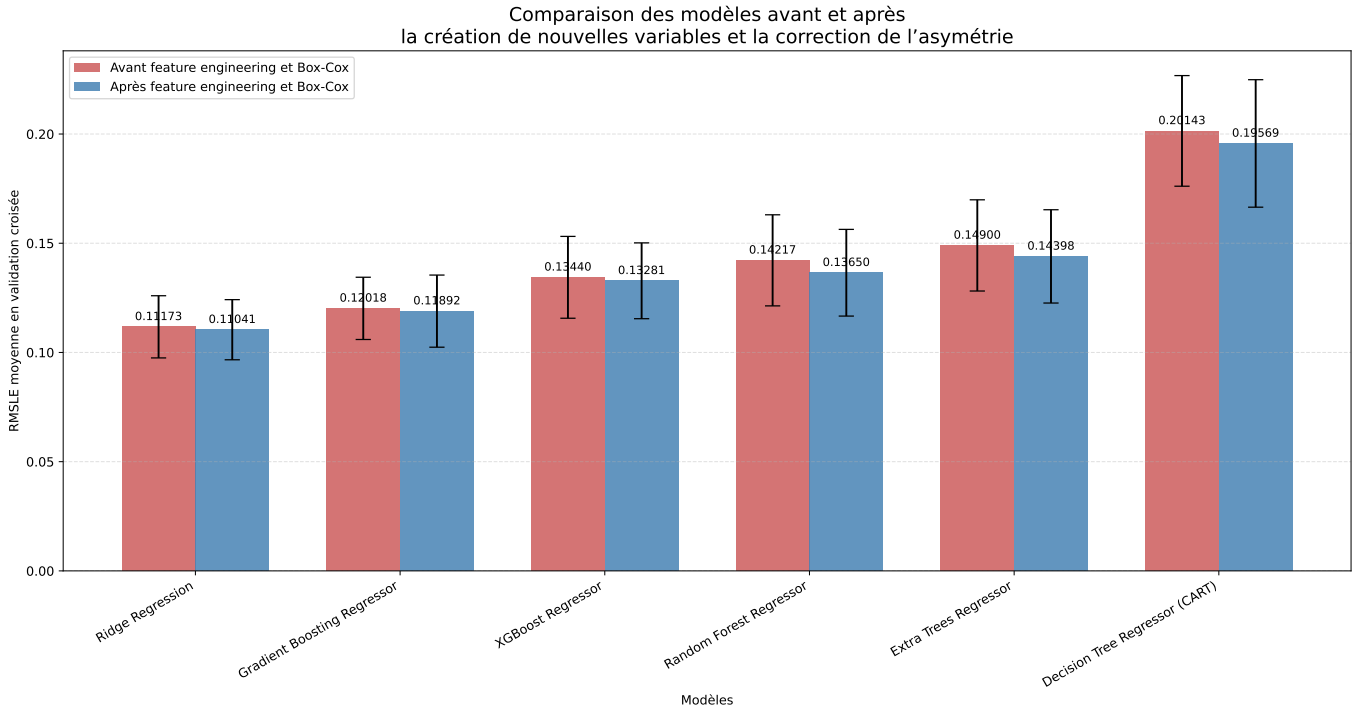


FIGURE 31 – Comparaison des performanes des modèles avant et après correction d'asymétrie et création de nouvelles variables

On peut noter pour chacun des modèles, une nette amélioration des performances de prédictions après la création des nouvelles variables et la correction de l'asymétrie. Ces ajustements supplémentaires sur les variables ont en effet aidés à améliorer les performances de prédictions des modèles.
Pour la suite de l'étude, nous gardons seulement les performances des modèles entrainés et évalués sur les données après la création des nouvelles variables et la correction de l'asymétrie. Les performances des modèles entrainés avant la création des nouvelles variables et la correction de l'asymétrie ne sont pas conservées.

IX. Optimisation des hyperparamètres

Les performances d'un modèle d'apprentissage automatique dépendent fortement des hyperparamètres utilisés lors de son entraînement. Contrairement aux paramètres du modèle, qui sont appris automatiquement à partir des données (par exemple les poids d'un modèle), les hyperparamètres doivent être définis avant l'entraînement. Leur choix influence directement la qualité des prédictions, la capacité de généralisation du modèle ainsi que son temps d'entraînement.

Le tuning d'hyperparamètres, ou optimisation des hyperparamètres, consiste à rechercher les valeurs d'hyperparamètres qui permettent à un modèle d'obtenir les meilleurs performances sur des données qu'il n'a pas utilisées directement pour apprendre.

Il existe plusieurs méthodes de tuning d'hyperparamètres

Grid Search :

On définit une liste précise de valeurs, comme par exemple :

- `n_estimators` $= [100, 300, 500]$
- `max_depth` $= [5, 10, 20]$

puis on teste toutes les combinaisons. Ici, cela donne $3 \times 3 = 9$ configurations.
C'est assez simple, mais ça peut vite devenir très couteux en calculs.

Random Search :

Pour cette méthode, on définit des plages, puis on tire aléatoirement un certains nombres de combinaisons.
On ne teste donc pas tout l'espace de recherche. Cela peut être plus rapide que Grid Search, mais les essais précédents ne guident pas nécessairement les suivants.

Optuna :

Optuna effectue une recherche plus intelligente. Au lieu de simplement tester les combinaisons, il utilise les résultats des essais précédents pour décider quelles zones de l'espace de recherche semblent les plus prometteuses.

C'est cette méthode d'optimisation des hyperparamètres que nous utiliserons dans cette étude.

IX.1. Principe de fonctionnement d'Optuna

Le fonctionnement d'optuna repose sur deux éléments principaux :

- une fonction objectif (objective function) ;
- une étude d'optimisation (study)

IX.1.1. La fonction objectif

La fonction objectif décrit principalement comment un ensemble d'hyperparamètres doit être évaluer. A chaque appel, elle reçoit de Optuna un objet trial représentant un nouvel essai d'hyperparamètres. A l'intérieur de la fonction objectif, plusieurs étapes sont réalisées :

1 - Proposition des paramètres à tester

Pour chaque hyperparamètre (par exemple le nombre d'arbres, la profondeur maximale ou le taux d'apprentissage), une plage de valeurs est définie. Optuna propose alors à l'objet trial une combinaison de valeurs appartenant à ces plages.

2 - Construction du modèle

Le modèle est ensuite créé en utilisant les hyperparamètres proposés pour cet essai (trial).

3 - Évaluation par validation croisée

Le modèle est entraîné à l'aide d'une validation croisée à 12 groupes (12-fold cross-validation). Les données sont divisées en douze groupes : à chaque itération, onze groupes servent à entraîner le modèle tandis que le groupe restant est utilisé pour l'évaluer. Cette opération est répétée douze fois et donc chaque groupe est utilisé une fois comme ensemble de validation

4 - Calcul de l'erreur moyenne

Les douze erreurs obtenues sont ensuite moyennées afin d'obtenir un score unique représentant la qualité des hyperparamètres testés.

A la fin de chaque essai, la fonction objectif renvoie l'erreur moyenne calculée à Optuna.

IX.1.2. Optuna

Optuna en revanche, pilote l'ensemble du processus d'optimisation en choisissant progressivement les valeurs d'hyperparamètres les plus prometteuses.
Une fois la fonction objectif définie, elle est transmise à Optuna grâce à la méthode study.optimize().

Tout comme la fonction objectif, Optuna réalise plusieurs étapes à chaque essai (trial),

1 - il choisit une nouvelle combinaison d'hyperparamètres à tester ;
2 - il exécute la fonction objectif avec cette combinaison ;
3 - il récupère l'erreur moyenne renvoyée par la validation croisée ;
4 - il mémorise simultanément les hyperparamètres testés et l'erreur obtenue.

De cette façon, au fur et à mesure des essais, Optuna construit un historique complet reliant chaque combinaison d'hyperparamètres à ses performances.
Il exploite ainsi les résultats des essais précédents afin d'orienter progressivement la recherche vers les régions de l'espace des hyperparamètres ayant produit les meilleures performances. Les zones peu prometteuses sont progressivement abandonnées, tandis que davantage d'essais sont

réalisés autour des combinaisons ayant donné les erreurs les plus faibles.
Cette stratégie permet généralement de trouver de très bonnes solutions avec un nombre d'essais relativement limité, réduisant ainsi considérablement le temps de calcul.

Dans cette étude, chaque modèle est optimisé indépendamment. Une étude Optuna distincte est donc réalisée pour chacun des modèles suivants :

- Decision Tree Regressor (CART) ;
- Random Forest Regressor ;
- Extra Trees Regressor ;
- Gradient Boosting Regressor ;
- XGBoost Regressor ;
- Ridge Regression

Pour chaque modèle, Optuna recherche les meilleurs hyperparamètres en utilisant la même procédure de validation croisée à 12 groupes.
Une fois l'optimisation terminée, les meilleurs hyperparamètres de chaque modèle sont conservés, puis les performances des différents modèles optimisés sont comparées. Le meilleur modèle est celui présentant la plus faible erreur moyenne de validation croisée.
Nous listons ci-dessous les plages de paramètres explorées pour les hyperparamètres des différents modèles :

Random Forest Regressor :

- `n_estimators` : [100, 150, 200, ..., 600]
- `max_depth` : [None, 5, 6, 7, ..., 20]
- `max_features` : ["sqrt", None, 0.5]
- `min_samples_split` : [2, 3, 4, 5, ..., 10]
- `min_samples_leaf` : [1, 2, 3, 4, 5, ..., 10]

Extra Trees Regressor :

- `n_estimators` : [100, 150, 200, ..., 600]
- `max_depth` : [None, 5, 6, 7, ..., 20]
- `max_features` : ["sqrt", None, 0.5]
- `min_samples_split` : [2, 3, 4, 5, ..., 10]
- `min_samples_leaf` : [1, 2, 3, 4, 5, ..., 10]
- `bootstrap` : [False, True]

Decision Tree Regressor (CART) :

- `max_depth` : [None, 5, 6, 7, ..., 20]
- `max_features` : ["sqrt", None, 0.5]
- `min_samples_split` : [2, 3, 4, 5, ..., 10]
- `min_samples_leaf` : [1, 2, 3, 4, 5, ..., 10]

Gradient Boosting Regressor :

- `n_estimators` : [100, 150, 200, ..., 600]
- `learning_rate` : [0.01,..., 0.30]

- `max_depth` : [None, 3, 4, 5, 6]

Extreme Gradient Boosting (XGBoost)

- `n_estimators` : [100, 150, 200, ..., 600]
- `learning_rate` : [0.01,..., 0.30]
- `max_depth` : [3, 4, 5, 6]
- `subsample` : [0.6,..., 1.0]
- `colsample_bytree` : [0.6,..., 1.0]

Ridge Regression

- `alpha` : [0.1,..., 15.0]

Chaque étude (study) avec Optuna réalise 100 essaies. Comme chaque essai comporte 12 groupes, cela représente :

- $100 \times 12 = 1200$ entraînements par modèle.

Or 6 modèles différents seront examinés, ce qui représente au final :

- $6 \times 100 \times 12 = 7200$ entraînements au total et l'équivalent en évaluations.

Ce qui peut être assez long.

En effet, le temps complet d'exécution du programme de recherche des meilleurs hyperparamètres avec optuna a été d'environ 7 heures 19 minutes 58 secondes :



FIGURE 32 – Temps d'exécution du tuning d'hyperparamètres pour chaque modèle

```
Temps total d'exécution : 26398.66 secondes (≈ 7 heures 19 minutes 58 secondes)
```

La figure précédente présente les temps complets d'exécution des programmes d'optimisation pour chaque modèle. Par exemple, le temps complet de tuning pour le random forest a été de 2h46min12s, 1h50min07s pour Extra Trees, 54 secondes pour Decision Tree Regressor, 2h19min16s pour Gradient Boosting Regressor, 22min47s pour XGBoost Regressor et enfin 40 secondes pour la régression Ridge.

Pour chaque modèle, les meilleurs hyperparamètres retenues ainsi que les erreurs de prédictions associées (rmsle) sont données ci-dessous :

```
Random Forest Regressor
========================================================================
Meilleure RMSLE : 0.12878
Meilleurs hyperparamètres :
 n_estimators : 600
 max_depth : 18
 max_features : 0.5
 min_samples_split : 3
 min_samples_leaf : 1

========================================================================
Extra Trees Regressor
========================================================================
Meilleure RMSLE : 0.12773
Meilleurs hyperparamètres :
 n_estimators : 350
 max_depth : None
 max_features : None
 min_samples_split : 7
 min_samples_leaf : 1
 bootstrap : False

========================================================================
Decision Tree Regressor
========================================================================
Meilleure RMSLE : 0.17357
Meilleurs hyperparamètres :
 max_depth : 20
 max_features : None
 min_samples_split : 10
 min_samples_leaf : 10

========================================================================
Gradient Boosting Regressor
========================================================================
Meilleure RMSLE : 0.11640
Meilleurs hyperparamètres :
 n_estimators : 500
 learning_rate : 0.07598663706815467
 max_depth : 3

========================================================================
XGBoost Regressor
========================================================================
Meilleure RMSLE : 0.11207
Meilleurs hyperparamètres :
 n_estimators : 400
 learning_rate : 0.04145356595588792
 max_depth : 3
 subsample : 0.6832596200443937
 colsample_bytree : 0.657063580759345
```

```
======================================================================
Ridge Regression
======================================================================
Meilleure RMSLE : 0.11014
Meilleurs hyperparamètres :
 alpha : 14.990407486310895
```

On constate une amélioration globale des erreurs de prédictions de chaque modèle après optimisation des hyperparamètres. L'optimisation des hyperparamètres a donc été bénéficique pour chaque modèle. Et en particulier elle a énormément bénéficé au modèle XGBoost Regressor qui s'est amélioré de presque 16% en terme d'erreurs de prédictions et est devenue meilleur que le modèle Gradient Boosting Regressor.

Modèle	RMSLE avant tuning	RMSLE après tuning	Amélioration	Amélioration (%)
Ridge Regression	0.110411	0.110143	0.000268	0.24 %
XGBoost Regressor	0.132809	0.112075	0.020734	15.61 %
Gradient Boosting Regressor	0.118918	0.116405	0.002513	2.11 %
Extra Trees Regressor	0.143980	0.127729	0.016252	11.29 %
Random Forest Regressor	0.136497	0.128777	0.007720	5.66 %
Decision Tree Regressor	0.195688	0.173570	0.022118	11.30 %

FIGURE 33 – Tableau des erreurs des modèles (rmsle) avant et après tuning

```
Rmsle gradient boosting avant optimisation : 0.11892

Rmsle XGBoost Regressor avant optimisation : 0.13281

Rmsle gradient boosting après optimisation : 0.116405

Rmsle XGBoost Regressor après optimisation : 0.112075
```

Le meilleur modèle après optimisation des hyperparamètres est la régression Ridge qui continue de ce fait d'occupée la première place sur la performance de l'erreur de prédiction.

```
======================================================================
MEILLEUR MODÈLE APRÈS OPTIMISATION
======================================================================
Modèle : Ridge Regression
RMSLE avant tuning : 0.110411
RMSLE après tuning : 0.110143
Amélioration : 0.000268
Amélioration en pourcentage : 0.24 %
```

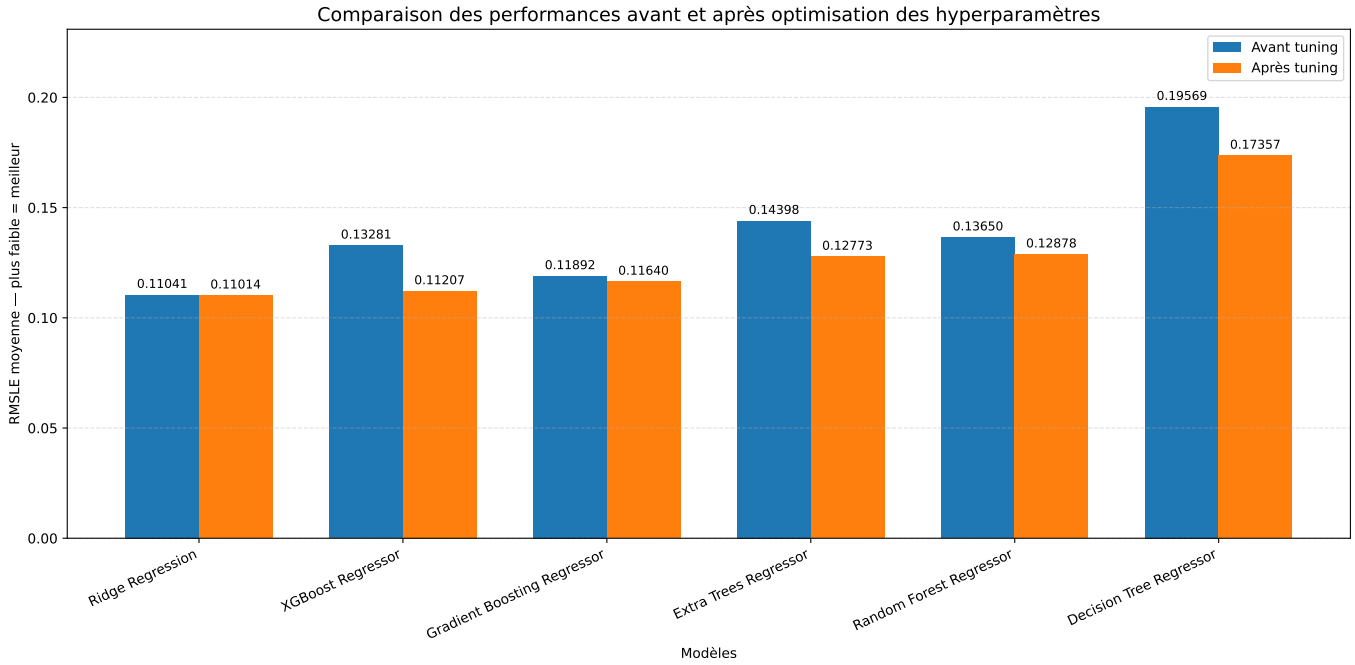


FIGURE 34 – Comparaison des erreurs des modèles (rmsle) avant et après tuning

Comme mentionné précédemment, on peut voir que : globalement, le tuning d'hyperparamètres améliore l'erreur de prédiction sur chaque modèle. Il a donc été bénéfique de l'effectuer.

Pour la suite de l'étude, nous conservons les 4 meilleurs modèles précédents avec leurs meilleurs hyperparamètres issues de l'optimisation. Il s'agit notamment de :

Ridge Regression avec

- alpha = 14.990407486310895

XGBoost Regressor avec :

- `n_estimators` = 400
- `learning_rate` = 0.04145356595588792
- `max_depth` = 3
- `subsample` = 0.6832596200443937
- `colsample_bytree` = 0.657063580759345

Gradient Boosting Regressor avec :

- `n_estimators` = 500
- `learning_rate` = 0.07598663706815467
- `max_depth` = 3

Extra Trees Regressor avec :

- `n_estimators` : 350
- `max_depth` : 15

- `max_features` : None
- `min_samples_split` : 7
- `min_samples_leaf` : 1
- `bootstrap` : False

Dans la partie suivante, nous utilisons les 3 modèles précédents les plus performants après Ridge (XGBoost Regressor, Gradient Boosting Regressor, Extra Trees Regressor) comme modèles de base et un modèle Ridge Regression (différent de celui obtenu dans cette partie) comme meta-modèle qui apprends la combinaison optimale sur les prédictions des modèles de base. C'est la méthode de stacking.

X. Stacking et évaluation finale des modèles

En apprentissage automatique, le stacking est une méthode d'ensemble qui consiste à utiliser plusieurs modèles de base pour effectuer des prédictions, puis à entraîner un dernier modèle afin qu'il apprenne à combiner leurs prédictions pour produire la prédiction finale.

X.1. Choix des modèles de base

On commence par sélectionner plusieurs modèles différents. Comme déclarer précédemment, on sélectionne les 3 modèles suivants obtenus en IX :

- Gradient Boosting Regressor ;
- XGBoost Regressor ;
- Extra Trees Regressor

Ces modèles sont appelés modèles de base ou base learners. Chaqu'un d'eux apprend à prédire le prix des maisons à partir des variables initiales du dataset : surface, quartier, nombre de salles de bain, année de construction, etc.

X.2. Obtention des valeurs prédites par validation croisée

Une fois les modèles de base sélectionnés, chaqu'un d'eux est entraîné par validation croisée K-Fold à 12 groupes.
Chaque modèle est entraîné sur 11 groupes et effectue des prédictions sur le 12ème groupe qu'il n'a pas utilisé pendant son entraînement. Le processus est répété 12 fois, en changeant à chaque fois le groupe utilisé pour la prédication.
On obtient donc 12 lots de prédictions pour chaque modèle, ou chaque prédiction est issue d'une observation (maison) que le modèle n'a pas vu pendant son apprentissage.
Ces prédictions sont appelées prédictions out-of-fold, ou prédictions hors pli. La validation croisée permet donc ici de produire des prédictions fiables qui serviront à entraîner le méta-modèle.

X.3. Construction du nouveau dataset

Une fois l'étape de prédictions des différents modèles de base terminée, les valeurs prédites sont ensuite regroupées dans un nouveau tableau.

Par exemple :

Prédiction Extra Trees	Prédiction Gradient Boosting	Prédiction XGBoost	Prix réel
201 000	202 500	204 000	203 000
177 500	178 500	179 000	178 000
245 500	247 000	248 000	247 500
319 500	321 000	322 000	321 500
163 000	164 000	165 000	164 500
288 500	290 000	291 000	290 500
228 500	229 500	230 000	229 800
198 500	199 500	200 000	199 700
271 500	272 500	273 000	272 800
344 500	346 000	347 000	346 500

Ces prédictions des différents modèles de base deviennent alors les nouvelles variables explicatives.
Le nouveau modèle sera entraîné sur ce tableau ; le prix réel restant la variable cible.

Prediction_Extra_Trees	Prediction_Gradient_Boosting	Prediction_XGBoost
11.883812	11.885455	11.890862
12.087420	12.071497	12.088419
11.829540	11.796750	11.817677
11.886438	11.944584	11.923014
11.924171	12.026637	11.970065
11.830022	11.782256	11.777859
12.069641	12.111347	12.076916
11.684867	11.626434	11.666289
12.116974	12.182289	12.139251
12.380035	12.374353	12.329226

FIGURE 35 – Aperçu de quelques prédictions (logarithmiques) des modèles de base

X.4. Entraînement du méta-modèle

On entraîne alors un nouveau modèle sur ces données. Ce modèle est appelé méta-modèle ou meta-learner. Dans notre cas, il s'agit d'une régréssion Ridge.
Le modèle de régréssion Ridge apprend comment combiner les prédictions des modèles de base

afin de se rapprocher le plus possible du prix réel.

Le paramètre alpha du modèle Ridge qui sert de méta-modèle au stacking ne peut pas être le même que celui obtenu par optimisation d'hyperparamètres.
Pour obtenir le meilleur paramètre alpha correspondant au méta-modèle Ridge, nous effectuons la même stratégie d'optimisation des hyperparamètres avec optuna mais cette fois sur les données qui sont les prédictions des modèles de base (du style du tableau de données précédent).

Nous obtenons `alpha` = 0.2862171020147687 (pour une RMSLE de 0.11430) qui sera utilisé pour construire le méta-modèle Ridge.

```
Best trial: 26. Best value: 0.1143: 100% 100/100 [00:10<00:00,  7.46it/s]
======================================================================
Ridge Regression
======================================================================
Meilleure RMSLE : 0.11430
Meilleurs hyperparamètres :
 alpha : 0.2862171020147687
```

FIGURE 36 – Recherche du meilleur paramètre alpha pour le méta-modèle

Pendant l'entraînement, le méta-modèle Ridge estime les coefficients associés aux prédictions des différents modèles de base. Il peut par exemple obtenir une combinaison de la forme :

$\hat{y} = 0,08 + 0,12 \times \hat{y}_{ETR} + 0,35 \times \hat{y}_{GBR} + 0,45 \times \hat{y}_{XGB}$

Il peut ainsi accorder davantage d'importance à certains modèles et moins à d'autres.

	Modèle de base	Coefficient appris
0	Prediction_Extra_Trees	0.177171
1	Prediction_Gradient_Boosting	0.261482
2	Prediction_XGBoost	0.580140

Constante apprise par Ridge : -0.22541352392503455

FIGURE 37 – Valeurs réelles des coefficients appris par le méta-modèle lors de l'entraînement

X.5. Réentraînement des modèles de base et prédiction d'une nouvelle maison

Après avoir terminer l'entraînement du méta modèle, chaque modèle de base est réentraîné sur l'ensemble complet des données d'entraînement.
Cette fois on ne conserve pas un groupe de validation. Cela n'a plus d'intérêt. La validation croisé à 12 groupes servait principalement à fournir des prédictions fiables sur lesquelles seraient entraînées le méta-modèle.
Lorsqu'une nouvelle maison arrive, ses caractéristiques sont transmises à chacun des modèles de base qui ont été réentraînes sur les données d'entraînement complètes. Chaque modèle produit

alors une prédiction. Ensuite les différentes prédictions sont transmises au méta-modèle Ridge. Lequel produit la prédiction finale grâce à la combinaison qu'il a apprise lors de son entraînement sur les prédictions des modèles de base.

XI. Évaluation finale des modèles sur l'ensemble de validation

Les modèles suivants sont tous évalués sur l'ensemble de validation :

- Ridge Regression Optimisé (obtenu en IX)
- Random Forest Optimisé (obtenu en IX)
- Stacking (obtenu en X)
- Gradient Boosting Optimisé (obtenu en IX)
- Extra Trees Optimisé (obtenu en IX)
- XGBoost Optimisé (obtenu en IX)

XI.1. Classement final sur l'ensemble de validation

```
======================================================================
CLASSEMENT FINAL SUR L'ENSEMBLE DE VALIDATION
======================================================================
 1. Ridge Regression Optimisé    RMSLE = 0.11186
 2. XGBoost Optimisé             RMSLE = 0.11350
 3. Stacking                     RMSLE = 0.11415
 4. Gradient Boosting Optimisé   RMSLE = 0.12128
 5. Extra Trees Optimisé         RMSLE = 0.13332
 6. Random Forest Optimisé       RMSLE = 0.13796
======================================================================
```

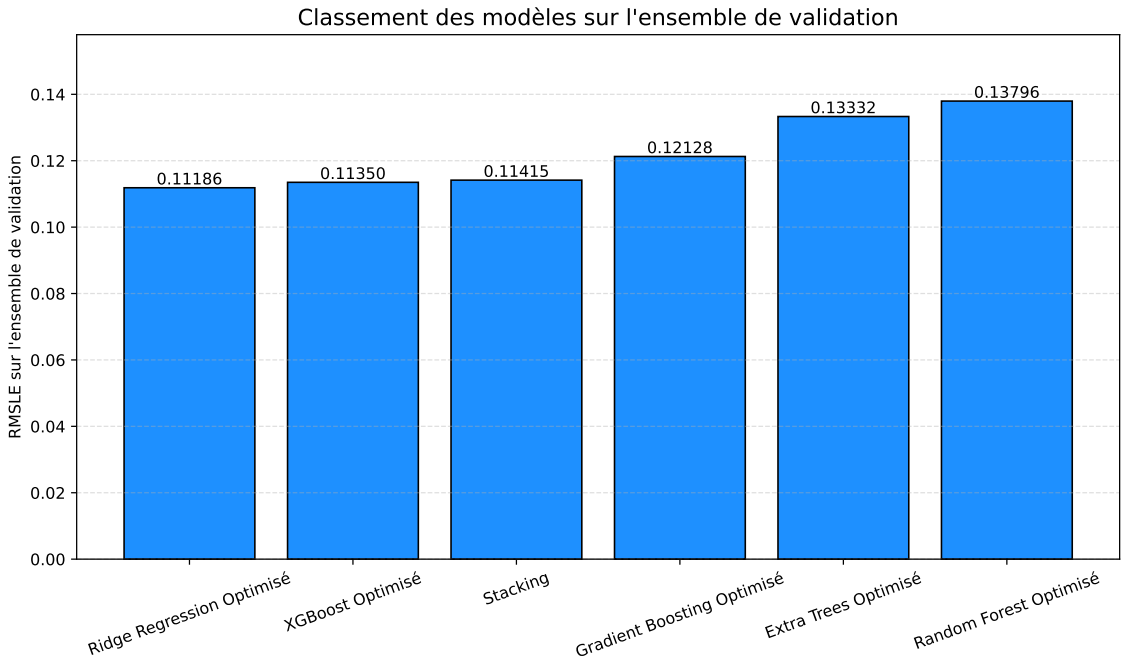


FIGURE 38 – Classement final des modèles sur l'ensemble de validation

Le modèle Ridge Regression optimisé obtient la meilleure performance sur l'ensemble de validation avec un RMSLE de 0,11186. Il présente à ce effet la plus faible erreur de prédiction parmi les modèles évalués. Ce qui montre une meilleure capacité de généralisation sur des données non utilisées lors de son entraînement.

Aussi, les écarts entre les trois premiers modèles (Ridge, XGBoost et Stacking) restent relativement faibles. Cela montre que ces approches offrent des performances comparables sur ces données et qu'aucun modèle ne domine très largement les autres.

Bien que le modèle de stacking combine les prédictions de plusieurs modèles de base, il n'obtient que la 3ème position. Il fait mieux que deux des modèles de base utilisés pour l'entrainer mais toujours moins bien que XGBoost et Ridge. Cela montre qu'une combinaison de modèles n'entraîne pas systématiquement une amélioration des performances par rapport au meilleur modèle prit individuellement.

Il est assez remarquable de constater qu'un modèle linéaire régularisé (Ridge) surpasse ici des modèles plus complexes comme XGBoost et le Stacking. Cela suggère que, sur ces données et après le prétraitement (gestion des valeurs manquantes et des valeurs aberrantes, transformation de Box-Cox, création de variables, etc.), la relation entre les variables explicatives et le prix des maisons est suffisamment bien capturée par un modèle linéaire régularisé.
Cela montre que la qualité du prétraitement peut parfois être plus déterminante que la complexité du modèle.

XI.2. Histogramme des prix réels et des prix prédits sur l'ensemble de validation

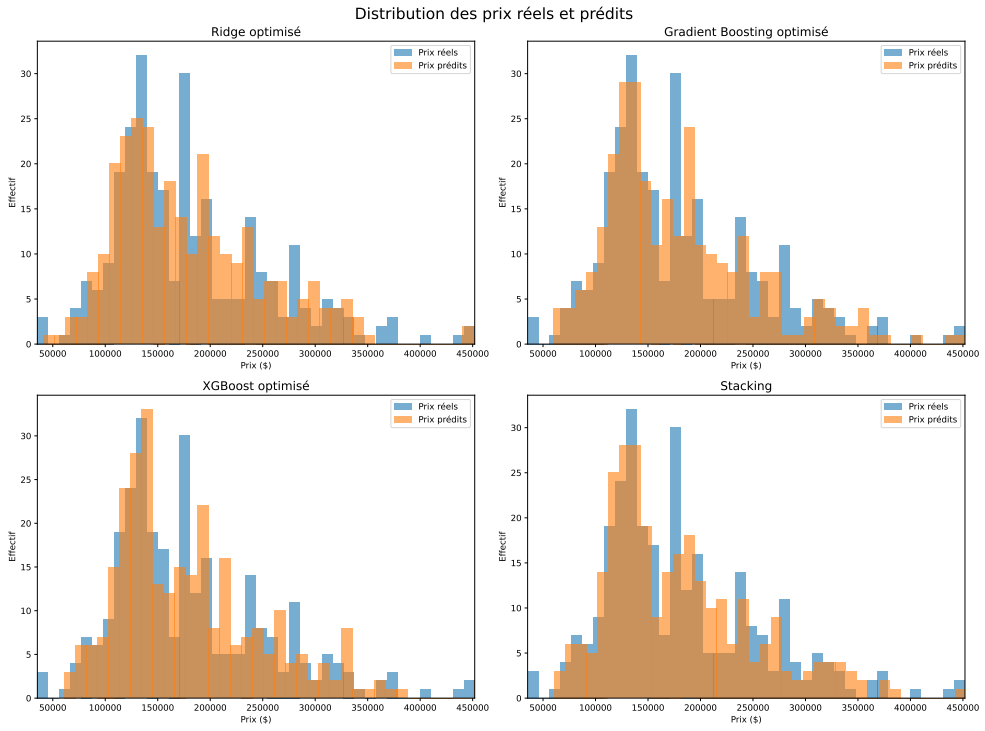


FIGURE 39 – Distribution des prix réels et des prix prédits pour les différents modèles

Les histogrammes montrent que, pour les quatre modèles, la distribution des prix prédits est globalement très proche de celle des prix réels, ce qui indique que les modèles parviennent à capturer la structure globale des données.

On observe néanmoins quelques différences, notamment dans les valeurs les plus élevées de prix de logements. Les biens les plus coûteux étant peu nombreux, ils sont naturellement plus difficiles à prédire avec précision.

XI.3. Nuage de points des prix réels et des prix prédits sur l'ensemble de validation

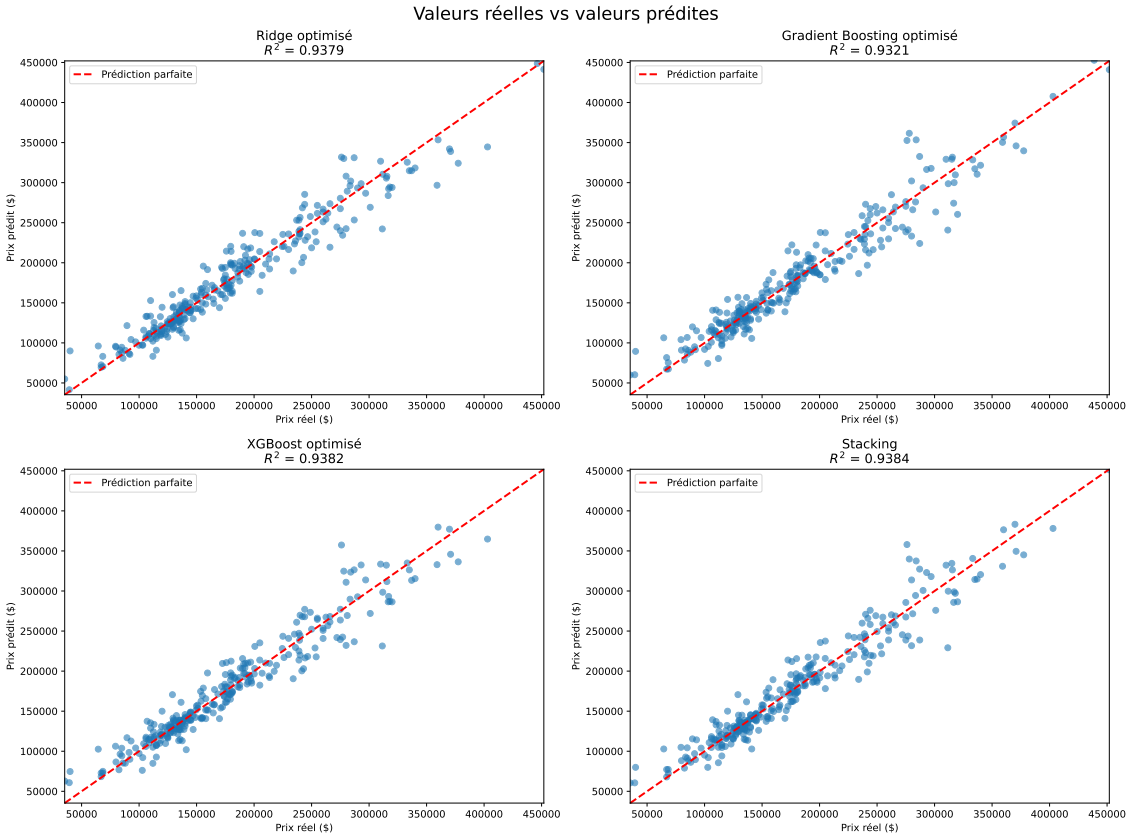


FIGURE 40 – Nuages de points des valeurs réelles et des valeurs prédites pour les différents modèles

Les nuages de points montrent que, pour les quatre modèles, les observations sont majoritairement concentrées autour de la droite de prédiction parfaite ($y = x$). Plus les points sont proches de cette diagonale, plus les prédictions sont précises.
Les coefficients de détermination $\mathbf{R}^2$, tous supérieurs à $0,93$, confirment que les modèles expliquent une très grande partie de la variabilité des prix des logements.

On remarque aussi que la dispersion des points augmente légèrement pour les maisons les plus chers. Les erreurs de prédiction sont donc un peu plus importantes pour les biens de forte valeur, ce qui est cohérent avec le fait que ces observations soient plus rares.

XII. Importance des variables

L'importance des variables est une mesure qui quantifie la contribution ou l'influence de chaque variable explicative sur les prédictions du modèle.
Son analyse permet donc d'identifier les variables qui influencent le plus les prédictions du modèle.

Dans le cas de notre étude, elle peut nous permettre de connaître quelles caractéristiques (variables) influencent le plus les prix des maisons.
Plus l'importance d'une variable est élevée, plus cette variable contribue aux prédictions du modèle. A l'inverse, une faible importance indique que la variable a peu d'influence sur les prédictions.

L'importance d'une variable est une mesure relative aux autres variables du modèle. Une variable importante dans un modèle peut ne pas l'être dans un autre.

Nous affichons ci-après le top 15 de l'importance des variables des 4 meilleurs modèles basés sur les arbres de décision :

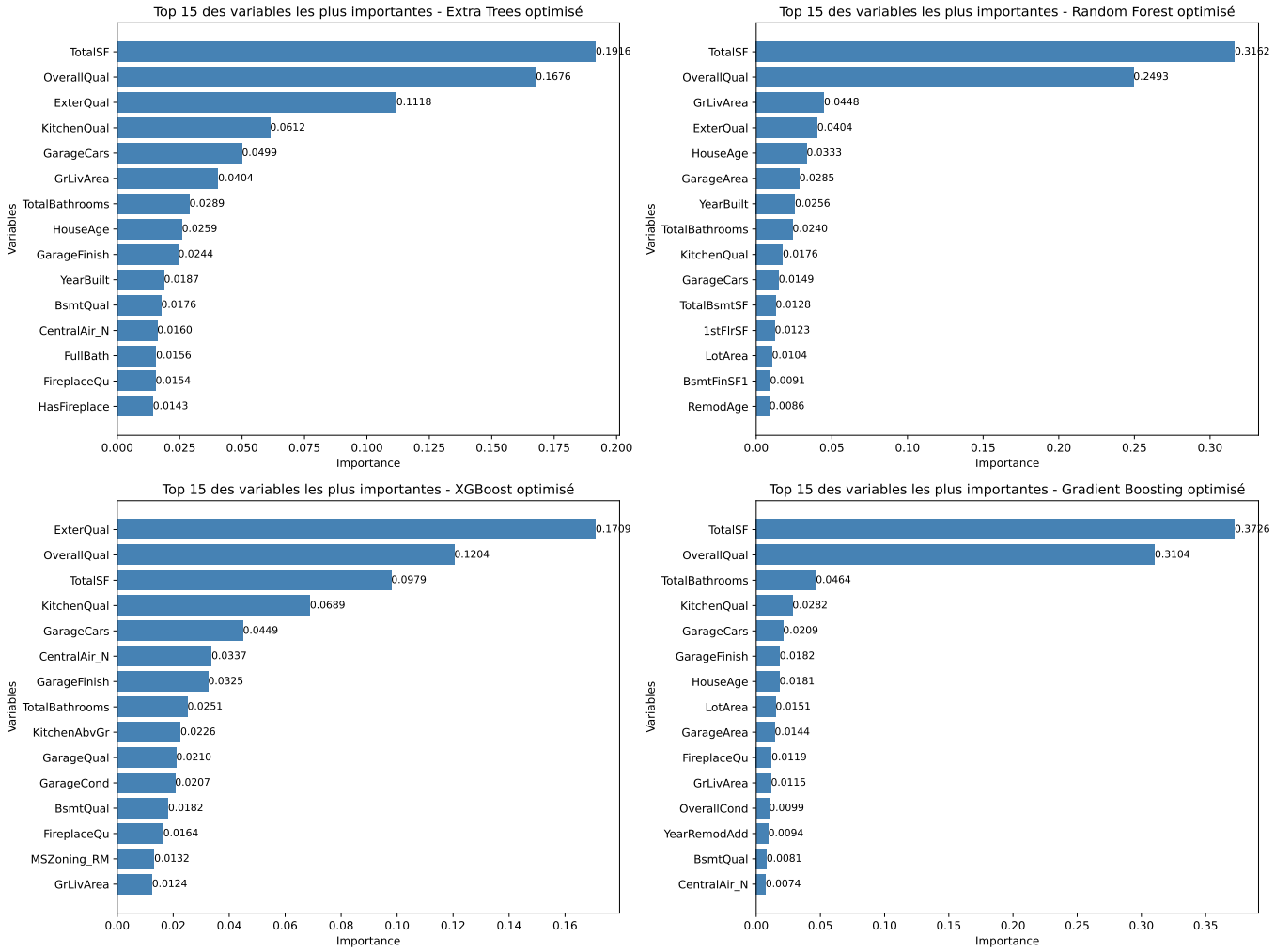


FIGURE 41 – Importance des variables

Pour la majorité des modèles précédents, on constate que les variables :

- TotalSF : surface total de la maison,
- OverallQual : qualité générale des matériaux et des finitions de la maison
- ExterQuall : État général de la maison

apparaissent comme les plus influentes dans la majorité des modèles précédents entraînées pour prédire le prix des maisons.
Néanmoins parmi les classement des 15 variables les plus importantes, dans chacun des modèles, plusieurs variables reviennent fréquemment, il s'agit notamment de :

- TotalBathrooms : nombre total de salle de bains de la maison
- GrLivArea : Surface habitable hors sous-sol
- HouseAge : Age de la maison
- KitchenQual : qualité de la cuisine
- GarageArea : surface du garage
- FirePlaceQual : qualité des ou de la cheminée

Ces résultats montrent que ces caractéristiques sont fortement associées aux prix prédits par les modèles. Lorsqu'elles sont modifiables (par exemple la qualité des matériaux et les finitions), elles constituent des leviers potentiels pour augmenter la valeur d'un bien immobilier.

Les variables crées lors de l'étape de Feature Engineering :

- TotalSF
- TotalBathrooms
- HouseAge

figurent parmi les plus importantes de plusieurs modèles. Cela montre que cette étape a permis de construire des variables plus informatives que certaines variables d'origine. Ce qui confirme une fois de plus l'intérêt du Feature Engineering dans l'amélioration des performances prédictives.
On peut également remarquer que la plupart des variables identifiées ici comme les plus importantes par les différents modèles figuraient déjà parmi les variables les plus fortement corrélées avec la variable cible *SalePrice* (voir VI.2). Cette cohérence entre l'analyse de corrélation et l'analyse des importances confirme que ces variables jouent un rôle majeur dans l'explication du prix des logements et qu'elles constituent les principaux facteurs exploités par les modèles de prédiction.

Références

[1] TensorFlow. *TensorFlow Decision Forests Documentation*. Disponible sur : https://www.tensorflow.org/decision_forests. (consulté en Mai 2026)

[2] Kaggle. *House Prices – Advanced Regression Techniques*. Disponible sur : https://www.kaggle.com/competitions/house-prices-advanced-regression-techniques. (consulté en Mai 2026).

[3] Doane, D. P., & Seward, L. E. *Measuring Skewness : A Forgotten Statistic ?* Journal of Statistics Education, vol. 19, no. 2, 2011.

[4] Google Developers. *Arbres de décision – Cours avancé de Machine Learning*. Disponible sur : https://developers.google.com/machine-learning/decision-forests/decision-trees. (consulté en juin 2026).

[5] Google Developers. *Forêts aléatoires – Cours avancé de Machine Learning*. Disponible sur : https://developers.google.com/machine-learning/decision-forests. (consulté en juillet 2026).

[6] Scikit-learn Developers. *Scikit-learn Documentation*. https://scikit-learn.org/ (consulté en juillet 2026).

[7] Optuna Developers. *Optuna Documentation*. Disponible sur : https://optuna.readthedocs.io/. (consulté en juillet 2026)

[8] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. *Optuna : A Next-generation Hyperparameter Optimization Framework*. Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '19), 2019, pp. 2623–2631. DOI : 10.1145/3292500.3330701.

[9] Optuna Developers. *Optuna : A Hyperparameter Optimization Framework*. Disponible sur : https://optuna.org/. (consulté en juillet 2026)